

Coordinate Transformation of CAD Drawings in Portugal

Geodetic foundations, DGT guidance and implementation with PT-TM06/ETRS89 (EPSG:3763)

The production application converts DXF/DWG drawings between 39 configured coordinate reference systems, with datum-specific GSB transformations, a live progress bar and estimated finish datetime, and a fitted CAD startup view. **Out-of-grid cleanup is enabled by default:** complete affected objects are omitted and recorded; valid objects continue through the full coordinate operation. Strict mode disables this cleanup. The input drawings are never changed.

This production revision corrects saved HATCH/MPOLYGON closing-point verification for geographic outputs such as EPSG:4326, using the unchanged metre-valued publication tolerance. It retains the ellipse sampler, bounded array processing and distinction between successful optional-coordinate cleanup and unresolved geometry. The application version remains **1.0**.

This release consists of `script.py` and `README.md`. Keep the existing GSB files beside the script or executable, or select their directory in Advanced settings. The application does not download grids and does not require the development test or benchmark scripts at runtime.

Abstract

The transformation of an engineering drawing between coordinate reference systems requires more than changing an EPSG identifier or translating its origin. It combines the interpretation of the source reference system, inverse cartographic projection, a datum transformation where necessary, forward projection and the reconstruction of CAD geometry. This paper documents a Python application for DXF/DWG reprojection, with particular attention to mainland Portugal and the target system PT-TM06/ETRS89, identified by EPSG:3763. It explains the geodetic relationships between ETRS89, Datum 73, Datum Lisboa, ED50 and the reference systems of the Portuguese archipelagos. The mathematical discussion covers ellipsoidal and geocentric coordinates, Transverse Mercator projection, Helmert and Molodensky transformations, polynomial models, NTV2 displacement grids, interpolation, uncertainty and geometric approximation. The implementation uses `pyproj`/PROJ for coordinates, `ezdxf` for DXF processing and an external ODA File Converter for DWG exchange. Particular emphasis is placed on distinguishing numerical precision, datum-model accuracy and CAD representation fidelity. Reproducible operating procedures, Windows installation commands, standalone packaging instructions and a bibliography complete the document.

Keywords: coordinate reference system; EPSG:3763; PT-TM06; ETRS89; Datum 73; Datum Lisboa; DGT; NTV2; CAD; DXF; DWG; geodetic transformation.

1. Purpose and scientific scope

1.1 The engineering problem

A CAD coordinate is a number with an implied spatial meaning. The point (10000, 10000) may represent metres in a national projection, coordinates in a local construction grid, or an arbitrary position in a drawing. Its numerical form does not identify its datum, projection, axis order or units. A reliable transformation therefore begins with documented source metadata.

The application is intended for drawings whose coordinates already belong to one of its supported reference systems. Its principal workflow is **EPSG:27493, Datum 73 / Modified**

Portuguese Grid, to EPSG:3763, ETRS89 / Portugal TM06. The user may select other supported source and target systems, including reverse transformations. Merely assigning the target EPSG code to an unchanged drawing is not equivalent to converting its coordinates.

In geodetic terminology, a *coordinate conversion* changes representation without changing datum, such as geographic to projected coordinates. A *coordinate transformation* changes datum through an empirically established relationship. A CAD reprojection can contain both operations. [IOGP, Guidance Note 7-2](#)

1.2 Implementation scope

Item	Behaviour
Main file and version	<code>script.py</code> , version 1.0
Default source CRS	EPSG:27493
Default target CRS	EPSG:3763
Native drawing processing	DXF with <code>ezdxf</code>
DWG processing	External ODA File Converter, with temporary DXF intermediates
Coordinate engine	<code>pyproj</code> , using PROJ and its CRS database
Interface	English Tkinter GUI and command-line interface
Output choices	DXF or DWG; one selected format for the complete batch
Incomplete objects	Omit unprocessable geometry, report it and continue by default; strict mode is optional
Main report	<code>survey_EPSG3763.json</code> , or <code><first-input-stem>_EPSG<target EPSG>.json</code>
Source identification	Declared by the user; not inferred automatically
Format-only conversion	Identical source and target EPSG codes are rejected

The mathematical treatment below explains both implemented operations and relevant background methods. The application does not expose every formula discussed as an independently selectable calculator. In particular, it does not implement a dedicated DGT polynomial calculator, a geoid conversion, a survey-network adjustment or a coordinate-epoch interface.

2. Reference systems in Portugal

2.1 DGT framework and territorial scope

The Direção-Geral do Território (DGT), Portugal’s Directorate-General for Territory, treats mainland Portugal, the Azores and Madeira separately in its reference-system documentation. Horizontal/cartographic systems and vertical systems are also distinguished. The mainland vertical reference is **Cascais Helmert 1938**, established from the Cascais tide-gauge record from 1882 to the end of 1938. The archipelagos use island-specific vertical references. [DGT, Reference systems](#)

For an engineering drawing, this distinction has a direct consequence: selecting EPSG:3763 establishes a horizontal reference system, but does not establish that the drawing’s Z coordinates are orthometric heights, ellipsoidal heights or elevations relative to a harbour datum. Those questions require separate source documentation.

2.2 PT-TM06/ETRS89: the main target

DGT states that PT-TM06/ETRS89 should replace the historical mainland systems. Its description relates ETRS89 to ITRS at epoch 1989.0 and to the stable Eurasian plate, and identifies the 1989, 1995 and 1997 international campaigns followed by GPS observation and adjustment of the mainland network. The PT-TM06 projection parameters apply from 2006. DGT also reports the EuroGeographics recommendation of Transverse Mercator for mapping at scales larger than 1:500,000. [DGT, PT-TM06/ETRS89](#)

The EPSG name is **ETRS89 / Portugal TM06**; **PT-TM06/ETRS89** is the familiar Portuguese system designation. The application uses **EPSG:3763** for this system.

Table 1. Projection and ellipsoid parameters for the principal target and source. Values are consistent with the DGT definitions and the EPSG records used by the application. [DGT, PT-TM06/ETRS89](#); [DGT, Datum 73](#)

Parameter	EPSG:3763	EPSG:27493
Datum/reference system	ETRS89	Datum 73
Ellipsoid	GRS80	International 1924 / Hayford
Semi-major axis, m	6,378,137	6,378,388
Inverse flattening	298.257222101	297
Projection family	Transverse Mercator	Gauss–Krüger / Transverse Mercator
Latitude of natural origin	39°40′05.73″ N	39°40′00″ N
Longitude of natural origin	8°07′59.19″ W	8°07′54.862″ W
Latitude of origin, decimal degrees	39.668258333333	39.666666666667
Longitude of origin, Greenwich degrees	−8.133108333333	−8.131906111111
Central-meridian scale	1.0	1.0

Parameter	EPSG:3763	EPSG:27493
factor		
False easting, m	0.000	+180.598
False northing, m	0.000	−86.990
Application X/Y convention	Easting / northing	Easting / northing

Negative coordinates are valid in systems with an origin inside the territory. A negative easting or northing is not evidence of a conversion failure. Conversely, plausible-looking positive values do not prove that the correct source datum has been selected.

2.3 Datum 73 and the historical mainland networks

Datum 73 followed reobservation of the mainland geodetic network in the 1970s. DGT identifies Melriça TF4 as its origin and describes the central location and single-block first-order adjustment. The system uses the Hayford ellipsoid and the false-origin values shown in Table 1. [DGT, Datum 73](#)

The origin offsets are part of its projection definition. Subtracting 180.598 m from X and adding 86.990 m to Y cannot, by itself, transform Datum 73 to ETRS89: the datum relationship and the target projection must also be applied.

Datum Lisboa has a different network history. DGT also describes the **Local Lisbon Triangulation**, established for 1:1,000 mapping, with a tangent-plane representation and coordinates of 12,000 m and 6,000 m assigned to Castelo de São Jorge. A drawing described only as “Lisbon coordinates” may therefore require more investigation than choosing a national EPSG code. [DGT, Datum Lisboa](#)

The Bessel-based historical system needs particular care. DGT notes that the historical Bessel–Bonne coordinates in use are not rigorously Bonne coordinates: they were derived by polynomial transformation from Hayford–Gauss Datum Lisboa. Its M coordinate is west-positive and P is south-positive. The application interface for EPSG:2963 and EPSG:5017 is **X = P, southing; Y = M, westing**. Historical source provenance and independent control points remain essential. [DGT, Bessel Datum Lisboa](#)

ED50 is a separate datum using International 1924, with its origin associated with Potsdam. The mainland projected representation included in this application is EPSG:23029, UTM zone 29N. Its transformation requires the ED50 relationship; neither a Datum 73 grid nor a Datum Lisboa grid is interchangeable with it. [DGT, ED50](#)

2.4 Azores and Madeira

DGT identifies PTRAO8-UTM/ITRF93 for the autonomous regions, linked to ITRF93 through the TANGO 1994 campaign and subsequent observations of the island networks. It recommends the UTM zone appropriate to the island group and replacement of the historical systems. [DGT, PTRAO8-UTM/ITRF93](#)

Table 2. Principal island projections.

Region	EPSG	UTM zone	Meridian
Western Azores	5014	25N	33° W
Central and eastern Azores	5015	26N	27° W
Madeira	5016	28N	15° W

These projections use GRS80, latitude of origin 0°, false easting 500,000 m, false northing 0 m and central-meridian scale factor 0.9996. The mainland default EPSG:3763 should therefore not be treated as a universal projection for every Portuguese island. The four mainland historical-datum grids distributed with this workflow are not island transformation grids.

2.5 Vertical references and geoid heights

For a compatible geoid model, ellipsoidal height h , orthometric height H and geoid undulation N_g are related by:

$$H = h - N_g.$$

DGT describes GeodPT08 for mainland Portugal, with 0.025° grid spacing, GRS80-referenced undulations and an estimated overall vertical accuracy of 4 cm relative to the national geodetic and levelling networks. This is a distinct dataset and operation from a horizontal NTV2 grid. [DGT](#), [Geoid model](#)

The application does not apply the height relation above. In ordinary horizontal conversions, it preserves CAD Z by default. Keeping a value such as 12.350 unchanged preserves that number's existing vertical meaning; it does not convert it to Cascais Helmert 1938, chart datum or ellipsoidal height.

3. EPSG systems available in the application

Both source and target selectors contain **39 unique complete CRS identifiers**, ordered as below. The catalogue follows the requested Global/Web, Mainland Portugal, Azores and Madeira grouping. Display labels are concise; PROJ uses the complete EPSG definition, not the descriptive label or comments. Source and target defaults remain **EPSG:27493** → **EPSG:3763** regardless of catalogue order.

3.1 Global / web standards

EPSG	Selector label	Application coordinate convention
4326	WGS 84 / Geographic 2D	X=longitude, Y=latitude, decimal degrees
3857	WGS 84 / Pseudo-Mercator	X=easting, Y=northing, metres
32629	WGS 84 / UTM zone 29N	X=easting, Y=northing, metres
32628	WGS 84 / UTM zone 28N	X=easting, Y=northing, metres

EPSG	Selector label	Application coordinate convention
32626	WGS 84 / UTM zone 26N	X=easting, Y=northing, metres
32625	WGS 84 / UTM zone 25N	X=easting, Y=northing, metres

The four WGS 84 UTM CRSs are separate from the corresponding ETRS89/PTRA08 projections even when they use the same UTM zone. A WGS 84 projection-to-projection conversion is not a datum change, so it does not need a mainland historical-datum GSB. A WGS 84 to historical mainland CRS still requires the correct grid stage. The realization/epoch warning includes these four WGS 84 UTM codes as well as EPSG:4326 and EPSG:3857. The projection definitions are recorded in PROJ's EPSG-derived database. [PROJ, Projected CRS definitions](#)

EPSG:3857 is offered for interoperability. Its metre coordinate unit does not make map distances suitable substitutes for local ground survey distances. The application does not assume that generic WGS 84 and an epoch-specific regional frame are identical for high-accuracy work.

3.2 Mainland Portugal

EPSG	Selector label	Application coordinate convention
4258	ETRS89 / Geographic 2D	Longitude/latitude, degrees
4937	ETRS89 / Geographic 3D	Longitude/latitude, degrees; ellipsoidal height, m
4936	ETRS89 / Geocentric coordinates	Earth-centred X/Y/Z, m
3763	ETRS89 / Portugal TM06	Easting/northing, m; default target
25829	ETRS89 / UTM zone 29N	Easting/northing, m
4274	Datum 73 / Geographic 2D	Longitude/latitude, degrees
27493	Datum 73 / Modified Portuguese Grid	Easting/northing, m; default source
4207	Lisbon / Geographic 2D	Greenwich longitude/latitude, degrees
20790	Lisbon / Portuguese National Grid	Easting/northing, m; Lisbon prime meridian
20791	Lisbon / Portuguese Grid	Easting/northing, m; Lisbon prime meridian
5018	Lisbon / Portuguese Grid New	Easting/northing, m; Greenwich prime meridian
4666	Lisbon 1890 / Geographic 2D	Greenwich longitude/latitude, degrees
2963	Lisbon 1890 / Portugal Bonne (X=South, Y=West)	Southing/westing, m; Lisbon prime meridian

EPSG	Selector label	Application coordinate convention
5017	Lisbon 1890 / Portugal Bonne New (X=South, Y=West)	Southing/westing, m; Greenwich prime meridian
4230	ED50 / Geographic 2D	Longitude/latitude, degrees
23029	ED50 / UTM zone 29N	Easting/northing, m

EPSG:5017 represents Lisbon 1890 / Portugal Bonne New with Greenwich-referenced longitude and south/west projected axes. The script uses the same south/west axis handling for EPSG:2963 and EPSG:5017 and routes both to the Lisbon 1890/Bessel grid family. Historical mapping-realization qualifications in Section 2.3 still apply. [PROJ, Projected CRS definitions](#); [PROJ, Conversion definitions](#)

3.3 Azores and shared PTRAO8 representations

EPSG	Selector label	Application coordinate convention
5013	PTRA08 / Geographic 2D	Longitude/latitude, degrees; Azores and Madeira
5012	PTRA08 / Geographic 3D	Longitude/latitude, degrees; ellipsoidal height, m; both archipelagos
5011	PTRA08 / Geocentric coordinates	Earth-centred X/Y/Z, m; both archipelagos
5014	PTRA08 / UTM zone 25N	Easting/northing, m; western Azores
5015	PTRA08 / UTM zone 26N	Easting/northing, m; central/eastern Azores
4664	Azores Oriental 1995 / Geographic 2D	Longitude/latitude, degrees
3062	Azores Oriental 1995 / UTM zone 26N	Easting/northing, m
4665	Azores Central 1995 / Geographic 2D	Longitude/latitude, degrees
3063	Azores Central 1995 / UTM zone 26N	Easting/northing, m
2188	Azores Occidental 1939 / UTM zone 25N	Easting/northing, m
2189	Azores Central 1948 / UTM zone 26N	Easting/northing, m
2190	Azores Oriental 1940 / UTM zone 26N	Easting/northing, m

The regional 1995 datums are distinct from PTRAO8. A projection change within one 1995 datum uses its own source/target CRS definitions. A change to PTRAO8 uses the avail-

able PROJ datum operation and retains its stated accuracy in the report. The mainland `pt73_e89.gsb`, `ptLX_e89.gsb`, `ptLB_e89.gsb` and `ptED_e89.gsb` files are not repurposed as island corrections. DGT identifies the same PTRAO8 geographic/geocentric identifiers for **both archipelagos**, not separate Madeira-only identifiers. [DGT, National EPSG code list](#); [DGT, PTRAO8-UTM/ITRF93](#)

3.4 Madeira

EPSG	Selector label	Application coordinate convention
5016	PTRA08 / UTM zone 28N	Easting/northing, m; Madeira region
25828	ETRS89 / UTM zone 28N	Easting/northing, m; separate datum, check area of use
4663	Porto Santo 1995 / Geographic 2D	Longitude/latitude, degrees
3061	Porto Santo 1995 / UTM zone 28N	Easting/northing, m
2942	Porto Santo / UTM zone 28N	Easting/northing, m

For geographic or geocentric PTRAO8 coordinates in Madeira, choose **5013**, **5012** or **5011**, already listed above. Repeating those same CRS definitions under another EPSG number would be incorrect. DGT identifies **5016** as the PTRAO8 UTM projection for the Madeira archipelago. **25828 is available for interoperability; it is not an alias or an automatically equivalent substitute for 5016.** The installed EPSG database’s area of use for 25828 does not include Madeira Island; a zone number alone is not sufficient to establish regional suitability. [DGT, National EPSG code list](#); [DGT, PTRAO8-UTM/ITRF93](#)

3.5 Coordinate conventions and axis order

Both selectors contain the same 39 complete CRS definitions. Geographic and geocentric PTRAO8 representations are shared by the Azores and Madeira and are listed once. The CLI and Python API use the same supported-code validation as the selectors.

The application requests `always_xy=True`: geographic input is CAD X=longitude and CAD Y=latitude, in decimal degrees. Native EPSG geographic definitions commonly list latitude first; importing a latitude/longitude table directly into CAD X/Y would reverse the coordinates. [pyproj](#), [Transformer API](#)

The Lisbon-meridian definitions in EPSG:20790, EPSG:20791 and EPSG:2963 are normalized through their CRS definitions before the grid is applied in Greenwich geographic coordinates. Do not add a manual longitude correction. EPSG:5017 already uses Greenwich but still has **X=southing, Y=westing**. Its unsupported PROJ method 9828 is represented by ordinary Bonne method 9827 plus the original south/west axes, as for EPSG:2963. This zero-offset mathematical equivalence is checked with a separate east/north Bonne reference. The 1995 island CRSs retain their own datum definitions and do not bypass any required-grid or missing-best-operation safeguard.

A supported selector entry means the application understands that CRS's coordinate contract. It does not certify every source/target pair, every region, epoch or vertical datum. Validate the operation and its coverage; inspect the report for unavailable resources, stated transformation accuracy and any declared approximations. The conversion report records the selected CRS interfaces, coordinate-operation stages and actual grid metadata.

4. Mathematical foundations

4.1 Notation

Symbol	Meaning	Usual unit
λ, φ	Longitude and geodetic latitude	Radians in formulas; degrees at the GUI/CLI geographic interface
E, N	Projected easting and northing	m
X, Y, Z	Geocentric Cartesian coordinates when explicitly identified as such	m
a, b	Ellipsoid semi-major and semi-minor axes	m
f, e^2	Flattening and squared first eccentricity	Dimensionless
ν, ρ	Prime-vertical and meridional radii of curvature	m
$h, H,$	Ellipsoidal height, orthometric	m
N_g	height and geoid undulation	
$P_s,$ P_t	Source and target projection operators	Coordinate mappings
$T_{s \rightarrow t}$	Datum transformation	Coordinate mapping
F, J_F	Complete coordinate mapping and its Jacobian	Mapping; derivative matrix
ε_s	Source-space curve faceting tolerance	Source drawing units

Angles in trigonometric equations are radians unless stated otherwise. A CAD entity's ordinary X/Y/Z properties must not be confused with Earth-centred Cartesian XYZ merely because the letters are the same.

4.2 Composition of a projected-to-projected transformation

For a source position x_s , the general horizontal operation can be written:

$$x_t = P_t(T_{s \rightarrow t}(P_s^{-1}(x_s))).$$

The inverse source projection recovers geographic coordinates on the source datum. The transformation then expresses the position in the target datum. The target projection produces the required planar coordinates. Unit conversions and prime-meridian changes are included where needed. If the datums agree, the middle operation may reduce to an identity or a representation adjustment. [IOGP, Guidance Note 7-2](#)

For the principal application workflow, the chain is:

EPSG:27493 → inverse source projection → Datum 73 geographic coordinates → NTv2 grid → ETRS89 geographic coordinates → PT-TM06 projection → EPSG:3763.

The intermediate geographic positions are essential to applying the grid correctly. The NTv2 correction is not a constant translation to add directly to projected CAD X/Y.

4.3 Ellipsoid geometry and geocentric coordinates

The geometric relationships begin with:

$$f = (a - b)/a, \quad e^2 = 2f - f^2.$$

$$\nu = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}, \quad \rho = \frac{a(1 - e^2)}{(1 - e^2 \sin^2 \varphi)^{3/2}}.$$

The forward geographic-to-geocentric conversion is:

$$X = (\nu + h) \cos \varphi \cos \lambda.$$

$$Y = (\nu + h) \cos \varphi \sin \lambda.$$

$$Z = [\nu(1 - e^2) + h] \sin \varphi.$$

The origin is at the Earth's centre; the X axis intersects the equator at Greenwich, Y intersects it at 90° E, and Z follows the north polar axis. Here h is ellipsoidal height. EPSG:4936 and EPSG:5011 require genuine three-dimensional Cartesian coordinates, not a conventional plan drawing with an arbitrary Z elevation. [PROJ, Geodetic to Cartesian conversion](#)

The inverse operation recovers latitude, longitude and ellipsoidal height using the selected ellipsoid. PROJ supplies these conversions; the application does not implement an independent geocentric solver.

4.4 Transverse Mercator and PT-TM06

Transverse Mercator is conformal: it preserves infinitesimal angles while scale varies spatially. It does not preserve every finite distance or area. The central scale, ellipsoid, natural origin and false coordinates jointly specify a particular projection. PT-TM06 and UTM belong to this family but use different parameters. PROJ documents a high-order Krüger formulation

for its precise Transverse Mercator implementation. The application delegates the projection calculation to PROJ. [PROJ, Transverse Mercator](#)

For explanatory purposes, define $A = (\lambda - \lambda_0) \cos \varphi$, $T = \tan^2 \varphi$, $C = e'^2 \cos^2 \varphi$ and $e'^2 = e^2/(1 - e^2)$. The familiar leading terms illustrate the nonlinearity:

$$E \approx E_0 + k_0 \nu \left[A + \frac{(1 - T + C)A^3}{6} \right].$$

$$N \approx N_0 + k_0 \left[M(\varphi) - M(\varphi_0) + \nu \tan \varphi \left(\frac{A^2}{2} + \frac{(5 - T + 9C + 4C^2)A^4}{24} \right) \right].$$

The meridional distance is $M(\varphi) = \int_0^\varphi \rho(\theta) d\theta$. The two Transverse Mercator series above are deliberately truncated illustrations, not the numerical implementation or a substitute for a complete projection algorithm. Their powers of longitude difference explain why a drawing-wide translation cannot reproduce a general projection change. Classical cartographic development is available in Snyder's working manual. [Snyder, Map Projections](#)

For a short measured line, a local grid scale factor k relates ellipsoidal and projected distance approximately by $d_{\text{grid}} \approx k d_{\text{ellipsoid}}$. Ground-to-grid reduction also depends on height. Consequently, a transformed CAD distance is not automatically a ground distance, and a dimension text value should not be treated as a new survey measurement without checking its meaning.

4.5 Three- and seven-parameter datum relationships

A three-parameter model accounts for a translation between geocentric origins. A seven-parameter Helmert model adds three small rotations and one scale parameter. In position-vector convention, its component form is:

$$X_t = T_X + (1 + \mu)(X_s - r_Z Y_s + r_Y Z_s).$$

$$Y_t = T_Y + (1 + \mu)(r_Z X_s + Y_s - r_X Z_s).$$

$$Z_t = T_Z + (1 + \mu)(-r_Y X_s + r_X Y_s + Z_s).$$

If the published scale is s parts per million, then $\mu = s 10^{-6}$. Rotation values stated in arcseconds must be converted to radians before insertion into these equations. Coordinate-frame convention reverses the rotation signs. The convention and transformation direction therefore belong to the parameter specification. [DGT, Bursa–Wolf formula sheet](#); [PROJ, Helmert transform](#)

A national best-fit parameter set describes broad differences between reference frames. It cannot reproduce arbitrary local deformation within a historical triangulation network. A displacement grid provides spatially varying corrections to address that limitation.

A time-dependent transformation can additionally use $p(t) = p(t_0) + \dot{p}(t - t_0)$ for its parameters. The application has no coordinate-epoch input. It should not be described as an epoch-aware transformation service for high-precision dynamic reference frames. [PROJ, Helmert transform](#)

4.6 Molodensky transformation

The Molodensky formulation expresses approximate changes directly in geographic coordinates, using geocentric translations and differences between ellipsoids. With $\Delta a = a_t - a_s$ and $\Delta f = f_t - f_s$, define:

$$Q = -\Delta X \sin \varphi \cos \lambda - \Delta Y \sin \varphi \sin \lambda + \Delta Z \cos \varphi.$$

$$B = \Delta a \frac{e^2 \nu}{a} \sin \varphi \cos \varphi + \Delta f \left(\frac{a}{b} \rho + \frac{b}{a} \nu \right) \sin \varphi \cos \varphi.$$

$$\Delta \varphi = (Q + B)/(\rho + h).$$

$$\Delta \lambda = \frac{-\Delta X \sin \lambda + \Delta Y \cos \lambda}{(\nu + h) \cos \varphi}.$$

$$\Delta h = \Delta X \cos \varphi \cos \lambda + \Delta Y \cos \varphi \sin \lambda + \Delta Z \sin \varphi - \Delta a \frac{a}{\nu} + \Delta f \frac{b}{a} \nu \sin^2 \varphi.$$

Target coordinates are obtained by adding these changes to the source latitude, longitude and ellipsoidal height. These equations reproduce the mathematical structure of DGT's formula sheet using the notation defined above. They require the source ellipsoid and consistent units. They are background theory here, rather than a separately exposed application mode. [DGT, Molodensky formula sheet](#)

4.7 Second-degree polynomial models

A planar polynomial can absorb smooth systematic differences through normalized source coordinates. To avoid confusing its normalization constants with ellipsoidal height or map scale, denote those constants by L_E and L_N :

$$u = (E - E_r)/L_E, \quad v = (N - N_r)/L_N.$$

$$E_t = a_0 + a_1 u + a_2 v + a_3 u^2 + a_4 uv + a_5 v^2.$$

$$N_t = b_0 + b_1 u + b_2 v + b_3 u^2 + b_4 uv + b_5 v^2.$$

DGT publishes this form together with fitted coefficient sets. The normalizing origin and scales are indispensable: coefficients cannot be transferred to unnormalized coordinates or to a different grid origin without altering the model. A polynomial is empirical and should not be extrapolated outside its intended system and region without validation. The present application uses its configured coordinate-operation chain rather than directly evaluating DGT's published polynomial tables. [DGT, Polynomial formula sheet](#)

5. Portuguese datum transformations and NTV2 grids

5.1 DGT grid construction and published validation

DGT distributes NTV2 grids for Datum 73 and Datum Lisboa to ETRS89. Its description specifies 1,129 GPS-observed national geodetic vertices, kriging to construct the grid, 72'' spacing, and validation against 130 additional vertices excluded from construction. The published results are shown below. DGT identifies this grid work as CC BY 4.0. [DGT, Mainland coordinate transformations](#)

Table 3. DGT’s reported grid validation statistics.

Source datum	RMSE, m	Maximum error, m
Datum 73	0.06	0.16
Datum Lisboa	0.09	0.30

These are dataset-validation statistics, not a guaranteed error for every input vertex. DGT also supplies polynomial, Molodensky and Bursa–Wolf alternatives; their parameters and reported residuals are available in its transformation parameter sheet. [DGT, Mainland coordinate transformations; IGP/DGT, Transformation parameters](#)

For reproducible mainland work, this README recommends using the DGT grids for these two datum families and recording the exact files. This is an implementation recommendation based on the official source and traceable validation; it is not a claim that DGT has certified this CAD application.

5.2 Published parameter methods in context

Table 4. Horizontal residual RMSE for DGT’s fitted parameter methods, in metres.

Values are component statistics from the published parameter sheet; they are not directly interchangeable with a single combined horizontal error statistic.

Method	73 E	73 N	Lisboa E	Lisboa N
Degree-2 polynomial	0.106	0.096	0.793	0.852
Bursa–Wolf	0.381	0.359	1.404	1.493
Molodensky	0.844	0.563	1.694	1.600

The polynomial adjustment/check sets differ between datums; the parameter methods and NTV2 grids also have different validation datasets. Table 4 illustrates why local network deformation matters, rather than providing a universal ranking at every site. [IGP/DGT, Transformation parameters](#)

Gonçalves’ CNCG 2009 paper independently discusses the grid approach, including 0.1° spacing and an independent sample of 147 geodetic points. Its reported longitude/latitude component standard deviations were 0.052/0.047 m for Datum 73 and 0.073/0.087 m for Datum Lisboa. The accompanying FCUP resource describes both its four-datum grid collection and the DGT grids. [Gonçalves, 2009; Gonçalves, Coordinate transformations in Portugal](#)

The DGT spacing is finer, but comparing 0.1° and 0.02° alone does not establish which model is more accurate at a particular location. The control points, source observations, interpolation

strategy and validation procedure also matter. A comparison of two models should use a common independent control dataset.

5.3 Grid construction and grid application are different operations

Kriging estimates a correction surface from irregularly distributed geodetic observations during grid preparation. The resulting NTV2 file stores values at regular nodes. During an ordinary conversion, PROJ reads those existing values and interpolates locally; it does not repeat the original network adjustment or kriging process.

For a point within a cell, let u and v be normalized cell coordinates between zero and one. Either correction component follows the bilinear rule:

$$\delta(u, v) = (1 - u)(1 - v)\delta_{00} + u(1 - v)\delta_{10} + (1 - u)v\delta_{01} + uv\delta_{11}.$$

The weights sum to one. At a corner the interpolated correction equals the stored corner value. Along an edge, the equation reduces to linear interpolation between the adjoining nodes. The surface is piecewise bilinear, so denser storage samples the correction field more closely without creating new independent observations. [Gonçalves, 2009](#)

If corrections have already been expressed in an east-positive longitude convention:

$$\lambda_t = \lambda_s + \delta\lambda(\lambda_s, \varphi_s), \quad \varphi_t = \varphi_s + \delta\varphi(\lambda_s, \varphi_s).$$

NTV2 longitude offsets conventionally use west-positive values. The binary-format convention and the geographic-coordinate convention must therefore be interpreted together. PROJ handles the format-specific signs; users should not negate grid contents or manually reverse longitude signs. [PROJ, Geodetic TIFF grid conventions](#)

5.4 Inverse transformation and coverage

For $q_t = q_s + \vec{\delta}(q_s)$, an illustrative inverse iteration is:

$$q_s^{(j+1)} = q_t - \vec{\delta}(q_s^{(j)}).$$

The correction is defined as a function of source position. Subtracting one correction evaluated at the target position is generally not the rigorous inverse. The application requests PROJ's inverse grid operation when converting from ETRS89 to a historical datum. [IOGP, Guidance Note 7-2](#); [PROJ, Horizontal grid shift](#)

The application uses mandatory grid stages. Missing files, incompatible datum headers, invalid grid payloads and loading failures stop the drawing conversion. A genuine grid-domain failure follows the selected **object-cleanup policy**: by default the complete affected object is omitted, its source handle and coordinates are recorded, and conversion continues. `--stop-outside-grid`, or strict mode, makes that domain error fatal. No out-of-grid coordinate is transformed with extrapolated shifts, clamped to the grid, left active in its source CRS or assigned an approximate fallback datum operation.

Coverage is checked in the **grid’s source datum**, after the inverse source projection and longitude normalization. An EPSG area-of-use rectangle, a CAD \$EXTMIN/\$EXTMAX header and the actual GSB node domain are different things. Coverage diagnostics identify the stage longitude/latitude, valid grid rectangles, original source WCS coordinates, entity type, handle, layer and layout. For skipped objects, `outside_grid_objects` records the owning entity and source/grid-stage coordinates; the operation record holds the grid bounds. Fatal errors also appear in the failure report and Log tab. The 1e-12-degree boundary comparison slack is numerical rounding tolerance only; it is not a licence to move drawing geometry or extrapolate the grid.

At a grid edge, a valid source point may map to a target point just outside the **unshifted** source rectangle. If PROJ’s inverse fails specifically because that target cannot seed an in-grid inverse evaluation, the application attempts a bounded numerical inverse of the same grid. Only trial seeds are constrained to the source rectangle; the original target is unchanged. Every forward evaluation is inside valid source coverage, and a candidate is accepted only when its forward image reproduces the target within 1e-12 degree. A target without a valid in-grid inverse source raises a grid-domain error and follows the object-cleanup policy. This is neither a lower-accuracy datum fallback nor clipping of the user’s coordinates.

Two non-geodetic false-failure paths receive separate handling. An ARC’s mathematical circle centre can lie outside the grid while its visible short arc lies inside; the local display-scale anchor is therefore taken on the actual curve. A HATCH can also contain a stale seed point unrelated to its valid boundaries. Only a failed optional seed lying outside the source boundary envelope, or a non-finite seed, is removed after all boundary coordinates have passed. Original seed coordinates and the reason are archived under `auxiliary_coordinate_repairs`; the report records `invalid_hatch_seed_removed`. Valid seeds still use the mandatory grid. A failed seed inside the boundary envelope is not discarded individually: a typed grid-domain failure follows the complete-object policy, while other coordinate-operation failures stop conversion. HATCH boundary/fill semantics are described in the [ezdxf HATCH reference](#).

5.5 Grid filenames required by script.py

The deployed files are `pt73_e89.gsb`, `ptLX_e89.gsb`, `ptED_e89.gsb` and `ptLB_e89.gsb`. The DGT downloads `D73_ETRS89_geo.gsb` and `DLX_ETRS89_geo.gsb` have been renamed to the first two names, without changing their binary contents. These four names are used in the source instructions, GUI requirements and Windows packaging command.

Table 5. Datum families and deterministic filename selection.

Datum family	Relevant EPSG codes	First choice in a directory	Backward-compatible alternative
Datum 73	27493, 4274	<code>pt73_e89.gsb</code>	<code>D73_ETRS89_geo.gsb</code>
Lisbon 1937 / Hayford	4207, 5018, 20790, 20791	<code>ptLX_e89.gsb</code>	<code>DLX_ETRS89_geo.gsb</code>
Lisbon 1890 / Bessel	4666, 2963, 5017	<code>ptLB_e89.gsb</code>	None
ED50	4230, 23029	<code>ptED_e89.gsb</code>	None

Names are matched case-insensitively. Within each resource directory, the canonical `pt*_e89.gsb` name has priority. The original DGT names remain fallback names for older

installations, not additional build requirements. An explicit grid folder is exclusive. Otherwise an external directory beside the script/executable is searched before embedded resources. A selected invalid canonical file is rejected; the program does not silently try a different file to hide its failure.

The complete binary structure is checked before use: overview and subgrid record lengths, source/target datums, angular units, ellipsoid axes, subgrid extents and spacing, dimensions, node counts, parent relationships, byte length and finite node values. Both byte orders are accepted. Millimetre rounding of the DGT overview’s ellipsoid minor axes is allowed. A wrong-datum grid renamed to a recognised name remains invalid. Grid-edge extrapolation is disabled, with only a $1\text{e-}12$ degree round-off allowance on the source footprint.

Table 6. Binary provenance of the selected grid data.

Selected data	Internal version	Spacing	Nodes	Bytes
DGT Datum 73, now pt73_e89.gsb	IGP2011	72 arcseconds (0.02°)	59,010	944,528
DGT Datum Lisboa, now ptLX_e89.gsb	IGP2011	72 arcseconds (0.02°)	59,010	944,528
FCUP ED50, ptED_e89.gsb	JAG08_01	360 arcseconds (0.1°)	2,501	40,368
FCUP Lisbon 1890, ptLB_e89.gsb	JAG08_01	360 arcseconds (0.1°)	2,501	40,368

The two former FCUP Datum 73/Lisbon grids used the same alias filenames but different binary data. Therefore **a filename is not an accuracy or provenance certificate**. The actual selected paths, versions and SHA-256 values are recorded in every report. For the renamed DGT Datum 73 file, the supplied Viana do Castelo report records SHA-256 54256060b00910d614fcf7d73c1c2514c90e6b389c750b6bfde46f7220358708; the renamed DGT Lisbon file checked in this revision has SHA-256 55fcfa790fa76994d937a7d90806ddd4e8994ca86cfb9865. No grid files are modified by this script update.

The inspected DGT grids have 281 latitude rows and 210 longitude columns, with a rectangular node domain of approximately 36.7638889° to 42.3638889° N and -9.9305556° to -5.7505556° longitude. A rectangular extent does not certify offshore or border accuracy. Retain the original downloads, DGT attribution and project validation records.

5.6 Search paths, priorities and operation selection

Configuration	Search behaviour
NTv2 grid folder or --grid-dir specified	Search that directory only
Running script.exe without an explicit folder	Search the executable directory before the packaged module/data directories
Running script.py without an	Search beside the script

Configuration	Search behaviour
explicit folder	
Expected grid appears in more than one default location	First matching file wins
Application started from a different working directory	Working directory does not become an implicit grid location

An explicit grid folder replaces default search locations; missing files are not silently taken from another folder. Avoid commas, double quotation marks and line breaks in a grid directory path. Spaces are supported.

For different historical datum families, the chain passes through ETRS89 and uses the appropriate forward source grid and inverse target grid. Within one historical family, only the necessary projection and meridian operations are applied. For other supported CRS combinations, **TransformerGroup** supplies available PROJ operations. The script records the selected operation and blocks a lower-accuracy substitution when PROJ reports that the best known operation is unavailable. The separate **Allow lower-accuracy installed operations** / `--allow-degraded` setting is an explicit override for these non-mandatory stages. **Allow ballpark datum transformations** / `--allow-ballpark` is a separate permission. Neither permission substitutes for a missing or incompatible required mainland grid or supplies an out-of-grid datum fallback. Skipping an out-of-grid object is a separate cleanup choice. PROJ networking is explicitly disabled for deterministic local-resource operation. The drawing’s estimated geographic extent is used as an area of interest for operation selection. When cleanup is enabled, an extent that cannot be interpreted because of outliers is not allowed to abort conversion before object-level checks; the normal CRS-area selection is used instead. This advisory extent is not a survey-control validation or a guarantee of accuracy across an entire rectangular extent. [pyproj](#), [Transformer API](#)

The local NTv2 stage converts geographic degrees to radians, applies **hgridshift**, and converts back to degrees. The horizontal grid hub is EPSG:4258, ETRS89 longitude/latitude. A same-datum projection does not receive an unnecessary grid correction. Non-grid geocentric operations explicitly promote their source/target CRS definitions to 3D. Historical-grid/geocentric combinations are rejected because no compatible height-datum model is supplied. These are coordinate-operation stages; they do not alter CAD layer names or determine the original survey datum.

6. Reprojection of CAD geometry

6.1 A coordinate mapping is not a general CAD object mapping

Let F be the complete source-to-target coordinate mapping. Transforming the centre of a circle does not fully transform the circle, because scale and orientation can vary across its extent. Similarly, one shared block definition cannot generally represent the exact nonlinear image of every insertion at different positions.

The application consequently uses several geometric strategies. Point-defined entities are handled through their defining locations; curves are faceted; supported composite objects are exploded; and some entities use a local affine approximation. “Transformed coordinates” therefore

does not mean that every original object type, constraint, width, payload or editing relationship is preserved exactly.

6.2 World coordinates and object coordinates

CAD entities can express geometry in a world coordinate system, an object coordinate system, or a block-local frame. Extrusion vectors, insertion matrices, rotations and scales affect the displayed positions. A transformation must act on the intended world positions rather than on local coordinates that happen to have similar numeric values.

The implementation uses `ezdxf` entity/path facilities to obtain world geometry where supported. SOLID and TRACE vertices are explicitly taken in world coordinates. Block and dimension handling inspects virtual geometry before explosion. These choices reduce coordinate-frame mistakes but do not turn an arbitrary proprietary object into fully accessible geometry.

6.3 Curves and source-space chord tolerance

For a circular arc of radius R and angular interval $\Delta\theta$, the maximum separation between arc and chord is:

$$e_c = R [1 - \cos(\Delta\theta/2)].$$

A chord-error requirement $e_c \leq \varepsilon_s$ gives, for $0 < \varepsilon_s < R$:

$$\Delta\theta \leq 2 \arccos(1 - \varepsilon_s/R).$$

For small angular intervals, $e_c \approx R\Delta\theta^2/8$. These elementary relationships explain why tighter faceting tolerances increase the vertex count. Ordinary 2D polyline bulges are sampled directly from their signed circular sweep and radius. CIRCLE/ARC entities and circular hatch edges are evaluated from their true circular geometry rather than a cubic Bézier surrogate. ELLIPSE and SPLINE use their native construction evaluators; other supported curve types retain the documented `ezdxf` path fallback. Hatch bulges use the same signed circular sampling as ordinary polylines.

A CAD bulge is the tangent of one quarter of the signed arc angle. The implementation evaluates circular samples in a local frame based on the segment chord, avoiding subtraction of a distant circle centre from large national-grid coordinates. The original segment endpoints remain explicit vertices. Small nonzero chords are not treated as coincident merely because their absolute coordinates are large. An arc requiring more than 1,000,000 facets is rejected with a diagnostic asking the user to check source units and tolerance; the requested tolerance is not silently relaxed.

The default tolerance is **0.01 source drawing units** for a projected source. If those units are metres, it represents a centimetre-scale source-curve faceting criterion. The default for a geographic source is **0.00000001 degrees (1e-8)**, applied consistently by the GUI, CLI and direct `ConversionJob` API when no tolerance is supplied. A degree tolerance does not correspond to a uniform metre tolerance: its east-west distance varies with latitude.

After faceting, the vertices are transformed and connected with straight segments. A source-space tolerance is not a certified target-space error bound. In a small neighbourhood, its effect

is related to the local derivative of F , while projection/grid curvature introduces an additional contribution. The software does not compute a global bound for that combined geometric error.

Straight LINE entities and straight portions of 2D/3D polylines, leaders and hatch boundaries are additionally refined for transformation curvature. For each source segment, the algorithm transforms the quarter, half and three-quarter positions and compares them with the target-coordinate chord. It bisects a segment when the maximum sampled deviation exceeds **0.01 m** by default, or its source-space length exceeds **1,000 m**. Geographic XY deviations are converted to local metric components using the ellipsoid's separate longitude and latitude factors; Z remains metric. The same check applies to the straight chords produced by source-curve faceting.

For latitude φ , with prime-vertical radius ν and meridional radius ρ , the local differential scales for degree-valued coordinates are:

$$s_\lambda = \frac{\pi}{180} \nu |\cos \varphi|, \quad s_\varphi = \frac{\pi}{180} \rho.$$

This is a local metric for the *sampling criterion*, not a constant metres-to-degrees coordinate conversion. The actual coordinate mapping always uses the complete CRS/grid chain. The GUI's **Adaptive reprojection deviation (metres)** and CLI `--reprojection-tolerance-m` control this additional criterion independently of source sagitta. A long LINE becomes a polyline only when intermediate output vertices are required. Caps on recursion and vertex counts prevent unlimited allocation; cancellation is checked while sampling. An edge crossing the longitude wrap is explicitly rejected for separate handling instead of silently drawing a world-spanning chord.

Quarter-point checks and the spacing cap are practical safeguards, not a proof of the continuous maximum error for every conceivable mapping. Source-curve approximation, local text/pattern mapping and the interiors of faces/meshes have separate limitations. Face and mesh vertices are transformed, but their surfaces are not adaptively retessellated. Neither tolerance should be represented as a universal final geodetic accuracy.

Ellipse parameter intervals and source sampling An ellipse is sampled directly from its world-coordinate centre and axis vectors:

$$p(u) = c + a \cos u + b \sin u.$$

A full ellipse is an explicitly stored one-turn interval, even when its start parameter is not zero. A small but nonzero interval is retained as a short elliptic arc. Relative `isclose()` tests must not turn either case into an empty curve. Equal raw start/end parameters are treated as a degenerate interval and reported, not silently expanded into an invented full ellipse. Wrapped partial intervals retain their counter-clockwise direction. Hatch ellipse edges preserve an explicit 360-degree span before conversion from polar angles to ellipse parameters. [ezdxf](#), [Ellipse](#)

With $A = [a \ b]$ and $r_{\max} = \sqrt{\lambda_{\max}(A^T A)}$, the sampler chooses parameter steps so that $r_{\max}[1 - \cos(\Delta u/2)]$ does not exceed the source chord tolerance. This follows by applying the linear map A to the unit-circle chord error. It bounds the ellipse faceting error in source units; it is not a bound on datum-model uncertainty. The normal target-space adaptive checks still follow. Tilted ellipses retain their three-dimensional source geometry, and the per-object vertex limit is unchanged.

6.4 Local affine approximation

For an anchor x_0 and a nearby displacement Δx :

$$F(x_0 + \Delta x) \approx F(x_0) + J_F(x_0)\Delta x.$$

This first-order expansion captures local translation, rotation, scale and shear. The neglected term grows with both object extent and spatial variation of the derivative. A single affine matrix therefore cannot guarantee an exact transformation of a large raster frame, text object, proprietary solid or other extended entity under a nonlinear mapping.

The script estimates the Jacobian with finite differences. Centred differences use an XY step of 0.00001 degrees for geographic sources and 0.25 m otherwise; the Z step is separately 0.25 m when a 3D operation is active. The derivative is no longer coupled to the user's curve tolerance. A derivative probe alone may switch to a one-sided difference at a grid boundary, and that occurrence is counted. Actual geometry points outside mandatory-grid coverage remain fatal. This step is a numerical derivative parameter, not a grid accuracy statement. The report lists objects that received local affine treatment.

6.5 Entity treatment

Table 7. CAD processing strategies and their consequences.

Entity group	Processing	Consequence
POINT	Transform location	Full selected point-coordinate operation
LINE	Transform and adaptively refine source segment	Remains LINE when no intermediate vertices are needed; otherwise polyline
3DFACE, SOLID, TRACE	Transform defining WCS vertices	Raw SOLID/TRACE vertex order is preserved; extrusion thickness is local; interiors are not warped
ARC, CIRCLE, ELLIPSE, SPLINE, HELIX	Sample source curve and refine target chords	Replaced by polyline; native circle/ellipse/spline evaluators avoid a cubic-circle surrogate
LWPOLYLINE, ordinary 2D POLYLINE	Transform/refine vertices; map segment widths	Closed flag and original native handles retained where representable
Fitted 2D POLYLINE	Materialise stored fitted arcs; local affine treatment without bulges	Original fit tags archived when arcs are materialised; fit-editing semantics are not retained

Entity group	Processing	Consequence
3D POLYLINE	Transform/refine segments	Explicit closure and coincident points retained; replacements can change handles when vertex counts grow
MESH, polygon/polyface mesh	Transform geometric vertices	Connectivity retained; curved target surfaces are not retessellated
LEADER	Transform/refine vertices; locally scale annotation sizes	Arrow size uses an entity override rather than changing the shared DIMSTYLE
MULTILEADER/MLEADER	Materialise native display components before reprojection	Avoids applying an unsupported non-uniform transform directly to the composite
HATCH, MPOLYGON	Sample/refine each boundary; transform seeds	Holes/path flags retained; local affine pattern mapping; non-horizontal results use explicit boundary-only fallback in best-effort mode
INSERT, DIMENSION	Preflight, explode and transform displayed components	Shared-block and associative editing semantics are not retained; dimension text is not recomputed as a new measurement
ACAD_PROXY_ENTITY	Decode supported saved graphics before reprojection	Native display snapshot only; original tags and unsupported appearance metadata are archived
TEXT, MTEXT, other transformable entities	Local Jacobian/affine transformation	First-order approximation is reported; glyphs and custom surfaces are not globally warped
VIEWPORT when paper-space conversion is selected	Local affine treatment	Layout/display relationships require inspection
Raster, underlay and OLE	Accessible placement/frame geometry only	Pixels and embedded payloads are not warped
External references	No independent recursive file conversion	Referenced files require separately managed conversion
GEODATA, XRECORD, other metadata	Retained where supported	Opaque embedded coordinates are not universally reprojected

Constant-elevation curve replacements generally become LWPOLYLINE entities. Varying-elevation or geocentric replacements use 3D POLYLINE entities. General graphic attributes, such as layer, colour and lineweight, are copied to replacements where supported; XDATA is copied where possible and source-entity provenance is added. This is not a promise of byte-for-byte preservation of all CAD data.

Ordinary lightweight and legacy 2D polylines receive dedicated handling. Their stored closed flag determines closure; no polygon vertex-count rule is applied to a CAD polyline. Single vertices, repeated vertices, short segments and two-vertex closed polylines are retained. Coordinates are first converted from the entity’s object coordinate system to world coordinates, including its elevation and extrusion direction. All sampled coordinates are transformed and validated before the original geometry is changed.

For a horizontal target representation, the native entity and its handle are retained. Legacy POLYLINE vertices also retain their original handles and attached data at the original vertex positions when extra arc samples are inserted. Widths are not retained as unchanged metre-valued numbers in a degree-coordinate drawing, and no single area-equivalent factor is applied to every segment. Let A be the local mapping from the source polyline’s OCS XY plane to target XY and let t be the unit source tangent. The local perpendicular width scale is:

$$s_w = \frac{|\det A|}{\|At\|}.$$

This follows from area scaling divided by along-segment scaling and remains valid for a locally sheared or anisotropic mapping. It is not generally $\|An\|$ or $\sqrt{|\det A|}$. Factors are evaluated at both ends of each output segment. A native constant width is expanded into explicit segment start/end widths; legacy default widths are similarly made explicit. Variable widths interpolate by source arc/segment fraction before local scaling. Zero-length segments retain their coincident vertices and use a local area-equivalent scale because no tangent direction is defined. The scaling also applies between different projected CRSs, not only between metres and degrees. Thickness is mapped locally at the first vertex, including the sign of the source extrusion. These choices preserve local styling, not an exact nonlinear transformation of the entire width or thickness envelope, and are identified in the report.

Fitted legacy polylines require a separate distinction. A circular bulge cannot generally survive unequal scaling in a native 2D POLYLINE, so a fitted object containing bulges is sampled before coordinate transformation. Curve-fit paths use both their original and inserted fit vertices. Where a spline-fit path has generated display vertices, only that display chain is used; its auxiliary spline-frame control points must not become visible connecting segments. The result is an ordinary polyline with the original closure and, where the target remains horizontal, its entity handle and retained display-vertex handles. Fit and tangent flags are cleared so a CAD application does not reinterpret the sampled geometry as a new fitted curve. Fitted polylines without bulges retain the reported local affine treatment. The underlying matrix restriction and vertex flags are described in the [ezdxf POLYLINE reference](#).

When fitted arcs are materialized, the JSON report records `fitted_polyline_faceted` and archives the original POLYLINE, VERTEX and SEQEND tags in `fitted_polyline_sources`. This includes the original fitting flags, tangents, stored coordinates, bulges and vertex XDATA; auxiliary control vertices are archived instead of inserted into the displayed output path. This is a source-data record, not preservation of the original fit-editing behaviour. References to other

database objects in those tags do not constitute a complete independent copy of the referenced objects. If a spline-fitted object has control points but no identifiable stored display chain, the object is omitted and reported by default. The converter does not invent connecting geometry. Optional strict mode blocks publication for that condition.

A varying-elevation or geocentric result without width or thickness can be represented by a 3D POLYLINE. If width or thickness would be lost by that representation, the object is omitted and reported by default. The same policy applies to a mapped thickness direction incompatible with a horizontal 2D polyline. Strict mode instead blocks publication when such an object cannot be represented.

Degenerate data receives explicit treatment. A zero-length segment retains its coincident vertices; an active nonzero bulge on that segment is cleared and reported because it defines no finite circular arc. An empty legacy POLYLINE is retained. A zero-vertex LWPOLYLINE cannot be serialized by the DXF writer and contains no geometry: it is omitted with a warning identifying its source handle, without adding fictitious vertices. Malformed CAD geometry and unrepresentable arcs follow the failed-object policy. Typed grid-domain errors follow the separate out-of-grid cleanup setting. Non-finite values or invalid geographic ranges raised by the coordinate engine, missing resources and other coordinate-operation failures are not converted into grid-cleanup warnings.

Proxy objects are a distinct case. An `ACAD_PROXY_ENTITY` represents a custom object whose proprietary internals cannot be transformed directly by `ezdxf`. Its saved proxy graphics may provide native display geometry suitable for extraction; this does not recover its original application behaviour. [ezdxf](#), [ACADProxyEntity](#)

The application checks the saved graphics stream before replacing a proxy, rejecting incomplete chunks, unsupported drawing operations and undecodable geometry. Supported snapshots become ordinary CAD entities and follow the normal reprojection handlers. Proxies in reachable block definitions are decoded in block coordinates before insertion transformations are applied. Unreferenced blocks and excluded paper-space layouts are not independently processed. A snapshot may reflect the display state saved by the authoring application rather than the full mathematical model of its custom object.

Proxy print and rendering attributes require separate treatment. Command 26 identifies a plot-style trait using two 32-bit fields: a selection mode and a resource index. The four modes are `ByLayer`, `ByBlock`, dictionary default and explicit plot style. Commands 34 and 35 describe material and mapper traits. These attributes do not supply drawing vertices. A material mapper controls the placement of textures on surfaces; it is distinct from a geometry transformation matrix.

The application accepts these appearance commands while continuing to decode and reproject their associated geometry. It validates command framing, the eight-byte plot-style record and its mode, and the 28-byte mapper record. The embedded material reference remains opaque. The complete original command bytes are retained in the source archive. Proxy resource indices are not guessed to be native DXF handles. A proxy lineweight outside the supported CAD range is replaced with **ByLayer** (-1) and recorded as an appearance warning; it does not prevent otherwise valid geometry from being converted.

Replacement entities inherit existing native plot-style and material references from their parent proxy. Plot-style references are also carried through subsequent curve faceting. Per-primitive

proxy plot styles, embedded material assignments and texture mapping are not reconstructed: the conversion report explicitly warns that printing or rendered appearance may differ. Inspect those properties in CAD before plotting or rendering. These appearance warnings do not count as unresolved geometry; unknown drawing operations, clipping commands and malformed geometric data still prevent strict-mode publication.

The original proxy DXF tags, source handle, graphics hash and decoded command names are retained in the report's `proxy_entity_sources`. Coordinates in this archive remain in their source frame; block-local records are labelled accordingly. The archive does not transform opaque application data or recursively copy external resources. Keep the original drawing as the authoritative source for the proprietary object and keep the report with the converted drawing. A decoded display snapshot is not a native, editable Civil 3D, Architecture or other application-specific object.

If saved graphics are absent, corrupt or unsupported, the default conversion omits the proxy from the processed geometry, records its type, handle and reason as `entity_omitted`, and continues with the remaining objects. Its original source coordinates are not deliberately left active in the converted output. Under strict mode, `proxy_unresolved` identifies the decoding problem and prevents publication. To recover the omitted content, open the original drawing in the CAD application that understands the custom object, convert or export it to standard CAD entities, and run the conversion again.

6.6 Best-effort conversion, composite preflight and strict mode

Before an INSERT or DIMENSION is destructively exploded, the application inspects its available geometry. Missing block definitions, recursive references, content that cannot be copied completely, inaccessible dimension geometry and XCLIP boundaries are treated as unresolved conditions. In the default best-effort mode, a composite that cannot be processed safely is omitted and identified in the report. The omission can cover the entire composite, not only one internal primitive. Other processable objects continue through reprojection.

The **Stop if a geometric object cannot be transformed** option is **off by default**. With this setting, an object-specific processing failure causes the affected object and any partial replacement geometry to be removed from the processed output. The report identifies the omitted entity type, source handle and reason, and the conversion continues. Keep the original drawing together with the report so that omitted content can be located and recovered. Completion with omissions is explicitly distinguished from completion without omissions.

Select **Stop if a geometric object cannot be transformed**, or pass `--strict` on the command line, when publication must be blocked by any identified unresolved geometric object. Strict mode still permits the documented curve faceting and local affine approximations. It is a completeness gate for identified failures, not a proof of exact geodetic or semantic preservation.

Best-effort processing applies to individual CAD objects. It does not bypass invalid job settings, unavailable or incompatible datum grids, non-domain coordinate-operation failures, cancellation, drawing read/write failures or output-verification failures. Those conditions still stop publication of the affected drawing. Source drawing files are unchanged.

Out-of-grid objects and obsolete origins. The **Skip objects outside mandatory grid coverage** option is on by default. It applies to an actual grid-domain failure, not to a hard-coded coordinate value or layer name. Consequently a standalone stale point at (0, 0, 0)

outside the selected grid is skipped, but a legitimate projected origin inside the grid is retained. A zero in a block-local definition is not tested before the insertion transformation, and polyface face-index records are not mistaken for geometric points. The software cannot determine from coordinates alone whether an out-of-grid object is useful or obsolete; it reports the omission rather than making that semantic claim.

The unit of omission is the **complete failed leaf object**. A polyline that reaches outside the domain is omitted as a whole; the program does not drop its bad vertex and connect the surrounding vertices, shorten a line, clip a boundary or move coordinates. Blocks, dimensions and multileaders are expanded into displayed components first, so a skipped child need not remove successfully converted siblings. Partial replacement entities and that object's intermediate accounting are removed before its omission is recorded. An output with no surviving finite model-space geometry is not published.

Policy	Grid-domain error	Missing/incompatible grid or another coordinate-operation failure
Default cleanup	Skip complete affected object; record it; continue	Stop conversion
Cleanup off / --stop-outside-grid	Stop conversion	Stop conversion
Strict / --strict	Stop conversion, even when cleanup is selected	Stop conversion

`skipped_outside_grid` is a per-entity-type subset of `omitted`, not an additional omission total. Each skipped object has an `outside_grid_object_skipped` issue and a structured `outside_grid_objects` record containing handles, layer, layout and the available source/grid-stage coordinate context. The first 20 omissions are also written to the live log; subsequent objects remain individually recorded in JSON, avoiding a slow, overloaded GUI log.

Paper-space geometry is excluded by default. Retained block definitions, layout objects and coordinate-bearing metadata should not be described as universally reprojected simply because the model-space geometry was processed.

6.7 Z-coordinate behaviour

Coordinates	Behaviour
Purely horizontal 2D source/target; Keep Z enabled	Preserve the source Z value
Purely horizontal 2D source/target; Keep Z disabled	Set transformed Z to zero
Either selected CRS has three or more axes	Pass XYZ through the coordinate operation; the Keep Z checkbox does not override it
Horizontal NTV2 correction	Supplies no geoid or vertical-datum

Coordinates	Behaviour
	correction
Geocentric source or target	Requires meaningful three-dimensional coordinates

Version 1.0 deliberately blocks direct combinations of a configured historical NTv2 datum and a geocentric target/source when the necessary ellipsoidal-height datum relationship is absent. The four horizontal grids must not be used to invent that relationship. An NTv2 horizontal grid also does not determine the change in ellipsoidal height between a historical datum and ETRS89. Passing XYZ through the coordinate chain cannot supply that missing relationship. Geocentric work requires heights consistent with the applicable ellipsoid and datum; unchanged historical ellipsoidal height is not automatically a rigorous ETRS89 height.

After conversion, **projected and geocentric** targets are marked with CAD \$INSUNITS=6, meaning metres. Geographic targets receive \$INSUNITS=0, because a single ordinary CAD linear-unit code cannot describe degree-valued XY with metric Z. The source/target EPSG identifiers and explicit axis/unit descriptions are retained in the drawing/report. Geocentric input is converted to its geographic counterpart when estimating the operation's area of interest; Earth-centred XYZ is never mistaken for longitude/latitude.

7. Accuracy, uncertainty and verification

7.1 Separate sources of error

Table 8. Quantities that should not be confused.

Quantity	Meaning	Limitation
Floating-point precision	Numerical representation and arithmetic	Accuracy of the original survey
Coordinate-operation numerical error	Numerical evaluation of a specified mapping	Correct choice of source datum
Grid spacing	Sampling interval of a stored displacement field	Independent observation density or a universal error limit
Grid validation statistics	Agreement at a stated test dataset	Guaranteed accuracy at every drawing location
Curve tolerance	Source-geometry faceting	Datum-transformation uncertainty
Local affine approximation	First-order transformation of an extended object	Exact nonlinear distortion throughout that object
DXF/DWG audit and entity counts	Readability and structural checks	Geodetic accuracy or preservation of all object semantics
Round-trip closure	Internal	Independent agreement

Quantity	Meaning	Limitation
	forward/inverse consistency	with surveyed control

For independent uncertainty contributions, a first-order covariance model is:

$$C_t \approx J_F C_s J_F^T + C_{\text{op}}.$$

Here C_s represents uncertainty of the source coordinates and C_{op} represents the operation model. Correlations require additional terms and cannot simply be discarded. This equation is an interpretation framework; the application does not estimate these covariance matrices. [JCGM, Guide to the Expression of Uncertainty in Measurement](#)

For a simplified one-dimensional independent-error illustration, source and transformation standard uncertainties (one-standard-deviation equivalents) of 0.03 m and 0.06 m give $\sqrt{0.03^2 + 0.06^2} \approx 0.067$ m. This illustrative calculation does not turn DGT's published RMSE into a site-specific uncertainty estimate or include CAD approximation error.

7.2 Independent control points

Select control points with a documented source position and an independently established target position. Points distributed around the drawing extent are more informative than several points concentrated in one corner. Check the datum, vertical interpretation, coordinate epoch where relevant, and the independence of the reference values before comparing results.

For point i , define residuals $v_{E,i} = E_{i,\text{calc}} - E_{i,\text{ref}}$ and $v_{N,i} = N_{i,\text{calc}} - N_{i,\text{ref}}$. Useful summaries include:

$$\text{RMSE}_E = \sqrt{\frac{1}{n} \sum_{i=1}^n v_{E,i}^2}, \quad \text{RMSE}_N = \sqrt{\frac{1}{n} \sum_{i=1}^n v_{N,i}^2}.$$

$$\text{RMSE}_{2D} = \sqrt{\frac{1}{n} \sum_{i=1}^n (v_{E,i}^2 + v_{N,i}^2)}.$$

Also examine mean residuals, the largest horizontal residual, the spatial pattern and any suspect control marks. A constant offset may indicate an origin issue; a rotated or position-dependent pattern can indicate axis, projection, datum or source-network problems. Residual interpretation requires the actual project context.

The software does not collect control-point pairs or automatically calculate these project validation statistics. Its JSON report documents the operation and CAD processing, while survey validation is a separate task.

7.3 Reproducible numerical example

Table 9. Illustrative coordinate calculations, EPSG:27493 to EPSG:3763. These are synthetic test coordinates, not surveyed accuracy-control points. Values were calculated using

pyproj 3.7.2, PROJ 9.5.1 and the DGT Datum 73 grid now deployed as `pt73_e89.gsb`. All coordinates are metres; horizontal conversion preserves Z.

Point	Axis	Source	Target
1	E	0.000000	0.343349
1	N	0.000000	-0.017373
1	Z	10.000	10.000
2	E	10000.000000	10000.017434
2	N	10000.000000	10000.039341
2	Z	25.000	25.000
3	E	-50000.000000	-50001.297927
3	N	100000.000000	99997.629952
3	Z	40.000	40.000

Unrounded coordinates returned to the source system with maximum component discrepancies below 2×10^{-9} m in this small calculation. This demonstrates numerical forward/inverse closure only. It is not evidence of nanometre survey accuracy, and applying the inverse to the rounded table values introduces additional rounding error.

7.4 File verification before publication

Before saving, the model-space startup camera is reset to a single orthographic WCS top view with an 8% fitting margin around the target XY bounds. Stale split-viewport aspect ratios and model-layout UCS references are not inherited, and regeneration is enabled. This is the stored-view equivalent of preparing Zoom Extents, rather than an automatic command executed in the user's CAD application. The receiving CAD window's aspect ratio and its treatment of saved view state remain viewer-dependent. [ezdxf](#), [Zoom Layouts](#)

The application writes a temporary DXF, reopens it, audits it and compares the processed model-space entity count. It additionally verifies finite display geometry, visibility and the prepared WCS top-view camera. Output extents, limits and startup view are based on target XY, not stale source coordinates or a preserved metre-valued Z range. Original layer visibility is retained. An input containing geometry that leaves no surviving model-space objects is not published as a successful empty drawing.

For exposed native coordinate types, verification compares **the positions that actually define the saved geometry**, rather than counting whichever optional DXF tags happen to be stored. Entity handles and types must match. Active text placement, polyline open/closed state, separate hatch boundary groups and checked mesh topology are verified. A positional difference exceeding **0.00001 m** (0.01 mm), a non-equivalent point-count change or an unmatched entity blocks publication. Geographic horizontal differences use the target ellipsoid; Z is compared in metres. This is a serialization check, not the source-curve tolerance, datum accuracy or a certification of every style, seed, opaque solid or application payload.

Large polylines. Ordinary WCS lightweight polylines use float64 array comparisons in bounded chunks. Every positional coordinate is compared; this is not a sample or a bounding-box-only shortcut. Closed state, bulges, finite values, geographic domains and the same neutral duplicate-closure rules remain checked. Tilted object coordinate systems and other entity types retain the general verifier.

Text alignment. A LEFT-aligned TEXT uses its insertion point; ordinary non-LEFT alignments use their alignment point. FIT and ALIGNED text use both defining endpoints. The verifier follows `get_placement()` in world coordinates for TEXT, ATTRIB and ATTDEF, and the insertion point for MTEXT. It does not call a removed or regenerated *unused* alignment tag a lost vertex. Changed active placement, alignment mode or plain text content still fails. The local affine anchor also uses the active placement, so a centred label's unused insertion field at the origin does not control its grid-domain test. [ezdxf, Text entity and placement API](#)

Closed hatch boundaries in degree-coordinate drawings. HATCH and MPOLYGON boundaries are paired before their terminal vertices are interpreted. A writer can store an explicit final closing point or rely on the closed flag. When one closed, straight boundary has exactly one extra terminal vertex, the verifier measures both (a) that terminal point's distance to its own first point and (b) its distance to the other representation's first point. Both must be **at most 0.00001 m**, and all remaining coordinates must subsequently match in their original order at the same threshold. Geographic XY is converted to an ellipsoidal distance with `Geod.inv`; preserved Z contributes a metre-valued difference. There is no raw-degree or relative-coordinate tolerance. Opposite longitude-seam representations and non-identical longitudes at a pole are not treated as neutral closing vertices of a straight CAD boundary. Equal-count boundaries retain every point in the comparison. [ezdxf, HATCH boundary paths](#); [pyproj, Geod API](#)

This narrowly defined comparison fixes a false rejection when two closing coordinates differ only by numerical round-off and a CAD writer removes one of them. It does not remove geometry from either drawing. Open paths, curved/bulged loops, missing interior vertices, changed loop classification, changed closure/fill modes and loss of holes are not accepted by this rule. Unknown reordering or a non-equivalent representation still fails. Ordinary LWPOLYLINE/POLYLINE entities retain their existing exact-duplicate closure rule for closed, unfitted, width-free and bulge-free paths.

Accepted differences are recorded in `output_verification.coordinate_comparison.serialization_normal`. For a hatch, `closure_checks` records the expected/reopened counts, which representation had the extra terminal point, both metre-valued distances, the terminal and counterpart coordinates, and the unchanged tolerance. The independently checked terminal-to-counterpart distance is included in the comparison's reported maximum and coordinate count; it is not hidden from the result.

A genuine saved-geometry failure still prevents publication and records the object type, handle, layer, layout, processing stage and available expected/reopened coordinates in `failed_file.output_verification.failure`. Hatch failures additionally include `target_crs`, `expected_hatch` and `reopened_hatch`: path flags, closure, endpoint gaps, elevation/extrusion and indexed boundary coordinates. Up to 256 vertices per small loop are recorded in full; larger loops retain the first and last 16 indexed records, and at most 32 loops are included. Completeness flags explicitly identify truncated diagnostic snapshots. These limits apply only to the JSON evidence, **not** to geometry processing or verification.

For DWG output, ODA converts the verified DXF to DWG and converts that DWG back to an isolated DXF for the same checks before publication. Neither DXF nor DWG output is accepted merely because a writer returned success. Opaque payloads and unrecognised representation changes remain outside the equivalence rules. The DWG workflow is implemented but was not executed in the Linux audit environment for this delivery.

The output entity count need not equal the source count: exploding blocks and replacing

curves changes structure. Verification concerns the expected processed document and its saved representation. The original source file is not overwritten.

7.5 Executed checks and acceptance scope

This saved-hatch verification revision was tested on **18 September 2026** using **Python 3.13.5 on Linux**, ezdxf 1.4.4 and pyproj 3.7.2. **166 regression tests passed without skips**: 125 retained CAD/CRS/grid/runtime cases plus 41 focused cases for metric hatch closure, preserved holes, altered vertices, topology, object-coordinate systems, Z elevations, reporting and publication. Retained fixtures explicitly select stop-on-grid-domain behaviour where that is the intended test, and reflect the existing generic CRS validation and canonical renamed-grid priority. The number is the suite executed for this revision, not a cumulative count of every previous development test file.

A synthetic hatch reproduces the report's boundary counts **31/39/3 before saving and 30/38/3 after writer normalization**. Its first boundary has a terminal point differing from the first point by one floating-point step in latitude. The previous source rejects it with the same **hatch boundary 1 count changed: expected 31, reopened 30 exception**. The corrected source verifies it with a maximum positional difference of approximately **1.06e-9 m**. This describes numerical round-off in a synthetic fixture, not survey accuracy or measured coordinates from the original project hatch.

Tests deliberately alter real corners, interior vertices, holes, closure flags, fill modes, bulges and Z values. Those cases remain blocked. A targeted pair of publication tests substitutes a simulated format bridge for ODA: equivalent closing-point normalization completes; real geometry loss produces a structured `verify_dwg` failure report and no output. These are **mocked writer-normalization tests, not native DWG or ODA execution**. Real DXF conversion, saving, reopening and startup-view verification were also exercised through the actual renamed Datum 73 grid.

The four deployed GSB files were independently checked at **4,800 positions** (1,200 each). All passed the validator's separate **0.00002 m** numerical comparison threshold; the largest independent grid discrepancy was approximately **1.01336e-5 m**. **144 analytical Mercator/geocentric control positions passed**. The saved-geometry publication threshold remains independently **0.00001 m**. The complete 1,482-pair matrix was not rerun to completion for this patch; no new all-pair result is claimed. The coordinate-operation and geometry-transformation implementations themselves are unchanged by this patch.

The independent grid checker implements its own NTv2 parser/interpolator and analytical Mercator/geocentric equations; other projection calculations share PROJ. These checks do not establish equivalent survey accuracy. The Tk selectors, all 39 selection callbacks, unchanged defaults, action buttons, progress bar and finish-time display were checked under Linux/Xvfb at **1366 × 768**. Source compilation and direct-scope duplicate-definition checks also passed.

What the supplied EPSG:4326 report establishes. `vcastelo_EPSG4326.json` records a source EPSG:27493 to target EPSG:4326 DWG job. The intermediate DXF passed verification; publication then stopped in `verify_dwg` at HATCH B8BBC9 on 000_Boias_Navegação, where boundary 1 changed from 31 to 30 records. Its four ELLIPSE objects were processed in this run, leaving one unresolved proxy (BBEA7F). The report contains the hatch counts, but not the original/reopened endpoint coordinates. It therefore does not establish whether this specific

missing record is a round-off closing point or an actual geometry change. The new diagnostic snapshots resolve that evidentiary gap in any further failure.

The complete original Viana do Castelo drawing, its actual hatch endpoints, a new Windows/ODA round trip and a rebuilt executable were **not** executed here. Rebuild from the corrected source, convert from the original drawing and retain the new report. A proxy with no usable display geometry remains an explicitly reported omission rather than an invented replacement.

7.6 Performance without reduced geometric precision

This patch adds a boundary-pair check only for saved HATCH/MPOLYGON verification and richer diagnostics only on failures. The existing float64 coordinate batches, exact-coordinate caches, bounded array verification, progress/ETA and fitted startup view are retained. No source-curve tolerance, target-chord criterion or publication tolerance was relaxed.

A three-repeat synthetic DXF benchmark used **12,050 entities and 43,000 native positions**, EPSG:27493 → EPSG:4326, and the actual renamed DGT Datum 73 grid. Median end-to-end time was **4.025 s for the preceding source and 4.261 s for this corrected source** in this run. That small measured increase is disclosed rather than presented as a speed improvement; the workload contains no hatches, and host/timing variation limits interpretation. The purpose of this revision is to correct verification, not to claim further acceleration. Timing includes reading/auditing, coordinate processing, view setup, writing, reopened-coordinate checks and JSON reporting. Generation and cross-version comparisons are outside the timer; run order alternated.

Every benchmark run retained **12,050 of 12,050 entities, with zero omissions**. All **43,000 native positions** agreed exactly between sources and after reopening, and the original file checksum was unchanged. The workload comprises 6,000 LINE objects, 6,000 POINT objects and 50 LWPOLYLINE objects with 500 vertices each. These are Linux synthetic DXF results, not predicted Windows/DWG timings.

The fast conversion path handles long straight, width-free, thickness-free WCS lightweight polylines with bounded float64 arrays. It evaluates **every endpoint and every quarter/midpoint/three-quarter probe** through the complete coordinate operation. The existing 1 km source-edge criterion and conservative target-tolerance margin remain in force. An edge needing subdivision, a nonstandard plane, nonzero widths or bulges, or a 3D CRS returns to the general handler **before any object mutation**. No vertex is decimated and no tolerance is relaxed. Width-aware and fitted-curve handlers remain available.

Large saved polylines are verified in bounded coordinate arrays instead of per-vertex Python object loops. Simple linear bounds use vectorized reductions; complex/width-bearing bounds retain their normal implementation. Existing exact-coordinate caches, deferred layout compaction, mandatory grid checks, failure replay and semantic verification are preserved. Native array access is isolated in one read-only adapter with an iterator fallback; updates still use the entity API. [ezdxf](#), [Lightweight polyline](#)

The supplied Viana do Castelo report attributes **241.49 s** to reprojection/view preparation and **142.56 s** to its two output verification stages, out of **482.32 s** total. Those observations motivated the targeted array paths. Reading, auditing, ODA and disk costs still remain, so the benchmark percentages must not be applied mechanically to that project drawing. Per-stage

`timings_seconds` remain in new reports.

8. Application workflow and graphical interface

8.1 Prepare the drawing and reference information

Identify the source EPSG code from survey records, project specifications or reliable control points. Confirm that the coordinates are in the units expected by that CRS. A CAD drawing expressed in millimetres must not be passed as metre-based national coordinates merely because its CAD unit setting can be changed. Changing `$INSUNITS` is not a rescaling operation.

Place the required grid files beside the script/executable, or select a complete grid folder in Advanced settings. For the normal Datum 73 to PT-TM06 conversion, use the renamed DGT file `pt73_e89.gsb`. Keeping all four correctly identified datum families available supports the other historical systems without repeated folder changes.

Install ODA File Converter when either the input or selected output is DWG. A DXF-to-DXF job does not require ODA.

8.2 Main controls

Table 10. Main GUI controls.

Control	Use
Open DXF / DWG...	Add one or several drawings
File / Type / Size list	Review inputs and select the drawing shown in the preview
Remove	Remove selected drawings from the batch
Clear	Remove all inputs
Source EPSG	Declare the actual coordinate reference system of the inputs
Target EPSG	Select the required output reference system
Validate transformation	Build/inspect the coordinate operation and required grids
Folder / Browse...	Select the output directory
Format	Select DXF or DWG for the whole batch
Suffix	Set the addition to each input stem, normally <code>_EPSG3763</code>
Convert drawings	Start the batch
Cancel	Request cooperative cancellation
Advanced...	Open numerical, data-handling and dependency settings
Preview / Log / Method	Inspect the input drawing, processing messages and method notes
/ help	

The interface reserves a left input/settings panel and a right preview/output area. Controls can

scroll when the available work area is small, including a 1366×768 desktop. Text sizing and layout respond to the available screen area rather than assuming one fixed large monitor.

8.3 Output format selection

Opening or selecting a DWG chooses **DWG (.dwg)**. Opening or selecting a DXF chooses **DXF (.dxf)**. The user can then change the selection manually. The selected format applies to **every drawing in the current batch**, including a batch with mixed input formats.

Every drawing in a batch must share the selected source EPSG and coordinate-unit convention. Mixed DWG/DXF formats are supported, but drawings in different source reference systems require separate batches. Source, target and numerical settings are common to the batch.

Selecting another input can change the selected output format again. Reloading the preview does not undo a manual format choice. Before starting a mixed batch, check the Format field after making the final input selection. There is no “keep each input format” option.

Changing the target EPSG resets the suggested suffix to `_EPSG<target>`. If a custom suffix is needed, set it after the final target selection. The application rejects planned output names that collide with an input or with another planned output.

8.4 Preview interpretation

The preview displays the **original selected input**, in model-space top view, on a black background. It is not an automatic transformed-output view. Use the mouse wheel to zoom, drag with the left mouse button to pan, and **Fit** or a double-click to show the extents. **Reload** reads the selected input again.

The renderer respects supported layer visibility, colours, blocks, text, curves and hatch boundaries. Raster images are represented by frames. For responsiveness, display processing is bounded to 35,000 primitives and 300,000 vertices and uses its own display approximation. Notices identify preview limits or unsupported display content. These display limits do not reduce the conversion’s processing scope.

The preview requires Pillow (PIL) as well as the ezdxf drawing frontend and Tkinter. Section 10 provides the complete installation and import checks. DWG preview also requires ODA to obtain a temporary DXF representation. The packaged build must collect these dependencies before it is distributed.

8.5 Advanced settings

Table 11. Advanced settings and defaults.

Setting	Default	Interpretation
Source curve sagitta	0.01 m / 1e-8 degree	Source-space curve sampling
Target chord deviation	0.01 m	Additional adaptive straight-segment refinement
Allow lower-accuracy installed operations	Off	Explicit override when the best known non-mandatory PROJ stage is unavailable
Keep Z elevations in	On	Preserve Z in ordinary 2D

Setting	Default	Interpretation
horizontal conversions		horizontal operations
Transform paper-space geometry	Off	Include layout geometry only when explicitly intended
Skip objects outside mandatory grid coverage	On	Omit the complete affected object and record its coordinates/handle; strict mode overrides
Stop if a geometric object cannot be transformed	Off	Omit unprocessable objects and continue; enable to require strict publication
Allow ballpark datum transformations	Off	Permit available approximate PROJ operations outside mandatory local-grid stages
Audit and recover damaged DXF	On	Attempt supported recovery and audit
Overwrite existing output files	Off	Require explicit choice before replacing CAD outputs
ODA File Converter executable	Automatic detection	Optional explicit executable path
NTv2 grid folder	Default search locations	An explicit directory is exclusive
Requirements	Information button	Show installation guidance

Validate transformation confirms that an operation can be constructed and its required grids loaded. It does not scan every drawing coordinate, establish full grid coverage or check survey-control accuracy. Coverage is checked when coordinates are actually transformed.

8.6 Typical GUI procedure

1. Start `script.py` or `script.exe`.
2. Open the drawing and inspect the source preview.
3. Confirm **Source EPSG:27493** and **Target EPSG:3763**, if those are the documented systems.
4. Choose the output folder and inspect the automatically selected output format.
5. Confirm the grid selection with **Validate transformation**.
6. Review Advanced settings, including the curve tolerance, Z policy and **Skip objects outside mandatory grid coverage**. Leave cleanup on to omit out-of-grid remnants with a report. Turn it off to stop on domain errors, or enable **Stop if a geometric object cannot be transformed** to require strict completion.
7. Select **Convert drawings** and follow the Log tab.

8. Open the completed drawing in the intended CAD application and review `survey_EPSG3763.json`, including omitted counts and the listed object handles and reasons.
9. Compare relevant control points, dimensions, layers, hatches and important complex objects.

8.7 Progress, elapsed time and estimated completion

The footer shows a live progress bar, percentage, current file/phase and full **estimated finish date and time in the running computer's local time zone**, with elapsed and estimated remaining duration. The estimate initially reads “calculating from completed work” and updates after enough progress has been observed. A tilde marks an estimate, not a completion guarantee. If a long phase reports no new work for 30 seconds, the clock remains live and the estimate is revised rather than displaying an expired promise.

Progress now advances **within a drawing**, including large polylines and exploded composite objects. Reading/auditing, geometry conversion, fitting the output view, writing, verification, ODA exchange and final publication occupy separate weighted phases. File sizes provide the initial weights for a multi-file batch. Composite children share their parent's remaining work budget, and displayed progress never moves backwards.

This percentage measures estimated completed work, not exact elapsed-time fraction. DXF reading, some audits and ODA do not expose internal completion counts, so the bar can hold during those phases while the elapsed clock continues. ODA's internal percentage is not fabricated. **100% is emitted only after output publication and successful report writing.** Cancellation or failure preserves the last achieved progress and shows the actual stop time. The converter remains cancellable at its cooperative checkpoints.

9. Command-line operation

Run without input paths to open the GUI. When input paths are supplied, `--output-dir` is required. The examples below use **Windows Command Prompt** and the virtual environment created and activated in Section 10. Run `call .venv\Scripts\activate.bat` in the project folder before using these examples.

9.1 Basic examples

Default Datum 73 to PT-TM06 conversion:

```
python script.py "survey.dxf" --output-dir "output"
```

DWG input with DXF output:

```
python script.py "survey.dwg" --output-dir "output" --format dxf
```

DXF input with DWG output and an explicit grid folder:

```
python script.py "survey.dxf" --output-dir "output" ^
  --format dwg --grid-dir "grids"
```

A batch with an explicit common output format:

```
python script.py "site_a.dwg" "site_b.dxf" ^
  --output-dir "output" --format dxf
```

Reverse conversion:

```
python script.py "survey_EPSG3763.dxf" ^  
  --output-dir "reverse" ^  
  --source-epsg 3763 --target-epsg 27493
```

If `--format` is omitted, the first input's extension chooses the output format for the entire batch. Quote paths that contain spaces. A relative `--grid-dir` is resolved from the command's working directory; this is an explicitly supplied path, distinct from the default grid search.

To require strict completion without identified unresolved geometry:

```
python script.py "survey.dxf" --output-dir "output" --strict
```

Without `--strict`, unprocessable objects are omitted and the remaining geometry is converted. Review the report before relying on the completeness of the output.

To disable grid-domain cleanup without changing the other best-effort CAD handlers:

```
python script.py "survey.dxf" --output-dir "output" --stop-outside-grid
```

For strict completion of all identified geometric objects, use `--strict`; it takes precedence over the cleanup setting. The normal command omits and reports out-of-grid objects. Completion with omissions returns exit code 0, so inspect the report's status and counters before treating an output as complete.

9.2 Complete option reference

Argument or option	Default / effect
inputs	One or more DXF/DWG paths; no inputs opens the GUI
--output-dir	Required for command-line conversion
--source-epsg	27493
--target-epsg	3763
--format {dxf,dwg}	First input's format if omitted
--suffix	_EPSG<target>
--curve-tolerance	0.00000001 for geographic sources; 0.01 otherwise
--reprojection-tolerance-m	0.01; target-chord sampling criterion in metres
--allow-degraded	Explicit lower-accuracy installed PROJ override; never bypass required GSB files
--transform-paper-space	Include paper-space geometry
--allow-ballpark	Permit available ballpark operations; does not bypass mandatory local grids
--stop-outside-grid	Disable the default grid-domain object cleanup; stop on an out-of-grid object
--strict	Block publication if geometry cannot be transformed; off by default
--allow-unresolved	Compatibility alias for the default best-effort omission policy; see Section 6.6
--overwrite	Permit replacing existing CAD outputs

Argument or option	Default / effect
<code>--oda</code>	Explicit ODA executable path
<code>--grid-dir</code>	Exclusive NTV2 directory
<code>--version</code>	Display version 1.0
<code>-h, --help</code>	Display command-line help

Keep-Z and DXF audit/recovery are enabled for normal CLI jobs; they have no CLI toggles. The internal ODA timeout is 900 seconds per converter invocation and is not exposed as a GUI/CLI setting. The Python `ConversionJob` interface contains these additional fields.

Completion, including completion with reported omissions, returns exit code 0; handled conversion failure returns 1; missing required CLI output-directory information returns 2. An exit code of 0 alone does not establish geometric completeness: inspect the report status and `omitted` counters. A GUI-oriented executable built with `--windowed` does not provide a normal console for help and progress text. Use the Python command or the console build described below when command-line output is needed.

10. Windows installation and virtual environment

10.1 Runtime and build dependencies

Pillow, imported in Python as `PIL`, is required for visualization. `pip` is the package installer. The preview uses the `ezdxf` drawing frontend with a custom Tk Canvas backend. That frontend imports Pillow even when raster images are represented by outlines. Installing only `ezdxf` and `pyproj` therefore does not establish that the preview can start.

Table 12. Complete component inventory for this application.

Component / package	Python import	Function / scope
Python, 64-bit, including Tcl/Tk	<code>sys</code> , <code>tkinter</code> , other standard-library modules	Source interpreter and GUI runtime; install on the build computer
<code>pip</code>	Run as <code>python -m pip</code>	Installs and checks packages; does not render drawings
<code>setuptools</code> , <code>wheel</code>	Build support tools	Maintain the environment's packaging support; not drawing renderers
<code>ezdxf==1.4.4</code>	<code>ezdxf</code>	Reads/writes DXF, represents CAD entities and supplies the drawing frontend
<code>pyproj==3.7.2</code>	<code>pyproj</code>	Supplies the PROJ interface,

Component / package	Python import	Function / scope
		coordinate operations and packaged PROJ resources
Pillow>=12.0,<13	PIL	Image utilities imported by the drawing frontend; explicitly install for this preview
numpy>=1.26,<3	numpy	Numerical arrays used by ezdxf; normally installed as its dependency
fonttools>=4.61,<5	fontTools	Font processing for CAD text; normally installed as an ezdxf dependency
pyparsing>=3.0,<4	pyparsing	Parsing facilities required by ezdxf
typing_extensions>=4.6,<5	typing_extensions	Runtime typing compatibility utilities required by ezdxf
certifi	certifi	Certificate bundle declared as a pyproj dependency; does not supply transformation grids
PyInstaller>=6.15,<7	PyInstaller	Produces the application executable; install on the build computer
pyinstaller-hooks- contrib	Hooks consumed by PyInstaller	Supplies maintained collection rules for third-party modules, data and binaries
Four correctly named NTV2 .gsb files	Data files, not modules	Historical mainland datum corrections;

Component / package	Python import	Function / scope
ODA File Converter	Separate native executable	supplied beside the source and embedded by the build below Required for DWG input, output and preview; not installed by pip

Pip resolves declared dependencies automatically; the installation command nevertheless lists every runtime package above explicitly. Core `ezdxf` does not declare Pillow as a mandatory base dependency. Installing `ezdxf[drawing]` would pull additional drawing backends, including libraries that this application's custom preview does not use. Matplotlib, PyQt, PySide and PyMuPDF are unnecessary for this implementation.

Tkinter, Tcl/Tk, `venv`, `ensurepip`, `argparse`, `json`, `pathlib`, `threading`, `ctypes` and the other standard-library components are provided by the Python installation. **Do not run `pip install tkinter`, `pip install PIL` or `pip install venv`.** Select Tcl/Tk and pip when installing Python; the image package's install name is Pillow.

The core packages are pinned to the API baseline used for application verification. Auxiliary version bounds permit compatible upgrades; they do not claim that every future package combination has been tested. The build record must retain the versions actually resolved by pip.

10.2 Create and activate the virtual environment

Place `script.py`, `README.md` and the four chosen grid files in a working folder, for example `C:\DOWNLOADS\cad-epsg-conversion`. Open **Windows Command Prompt** (`cmd.exe`) there. The following are shell commands, not Python statements for the `>>>` prompt:

```
cd /d "C:\DOWNLOADS\cad-epsg-conversion"
py -m venv .venv
call .venv\Scripts\activate.bat
python -c "import sys; assert sys.prefix != sys.base_prefix"
python -c "import sys; print(sys.executable)"
```

The printed path should end in the project's `.venv\Scripts\python.exe`. Python creates the virtual environment; its activation script selects that environment in the current command shell. Activation usually adds `(.venv)` to the prompt. Every subsequent `python -m pip` command then targets this environment rather than an unrelated global installation. The `call` form also works when these commands are saved in a Windows batch file.

The `py -m venv .venv` command uses the interpreter selected by the Windows Python launcher; it does not force a particular minor version. Confirm it with `python --version` after activation. If `py` is unavailable, install the Python launcher or call the intended Python executable by its full path. An existing environment can be activated without creating it again.

10.3 Bootstrap pip, upgrade installation tools and install every runtime package

Run the following in the **activated environment**:

```
python -m ensurepip --upgrade
python -m pip install --upgrade pip setuptools wheel
python -m pip install --upgrade ^
    "ezdxf==1.4.4" "pyproj==3.7.2" "Pillow>=12.0,<13" ^
    "numpy>=1.26,<3" "fonttools>=4.61,<5" "pyparsing>=3.0,<4" ^
    "typing_extensions>=4.6,<5" certifi
python -m pip check
```

ensurepip bootstraps pip from Python's bundled resources; it does not obtain the newest pip release from the internet. The following **pip install --upgrade pip setuptools wheel** command upgrades the installation tools from the configured package index. Normal dependency installation requires access to that index or an equivalent local wheel repository.

--upgrade requests the newest available version allowed by each requirement. The exact **ezdxf** and **pyproj** pins retain their specified versions; the bounded supporting packages can advance within their ranges. Quotes are necessary around requirements containing **<** or **>** because Command Prompt otherwise interprets those characters as redirection. Each caret **^** must be the last character on its line, with **no trailing spaces**.

Use a Python version and architecture for which the selected packages provide compatible Windows binary wheels. If pip unexpectedly tries to compile a native package and fails, confirm the interpreter version and 64-bit architecture, upgrade pip, and check whether compatible wheels exist for the pinned packages. A compiler should not be assumed to repair a mismatched environment.

10.4 Check the visualization stack and start the program

pip check validates declared dependency relationships. It cannot detect an undeclared optional dependency such as **Pillow** in the base **ezdxf** installation. Import the **drawing frontend**, in addition to the top-level packages. Each import command must finish without an error; the following commands then print the installed versions:

```
python -c "import tkinter, ezdxf, pyproj"
python -c "import PIL, numpy, fontTools"
python -c "from ezdxf.addons.drawing import Frontend"
python -c "from ezdxf.addons.drawing import RenderContext"
python -c "import ezdxf; print('ezdxf', ezdxf.__version__)"
python -c "import pyproj; print('pyproj', pyproj.__version__)"
python -c "import pyproj; print('PROJ', pyproj.proj_version_str)"
python -c "import PIL; print('Pillow', PIL.__version__)"
python -m tkinter
python script.py
```

Close the small Tkinter demonstration window before proceeding to **python script.py**. In the application, open a representative DXF containing lines, arcs, text and hatches. Confirm that model space appears in the black preview, then test pan, zoom and Fit. A successful import alone does not verify that actual drawing content renders correctly.

For the coordinate runtime's diagnostic information:

```
python -c "import pyproj; pyproj.show_versions()"
```

Check the selected transformation through **Validate transformation**, then convert a drawing with known survey control. Python package checks do not establish that a source CRS is correct, that the proper grid was selected, or that ODA is installed.

10.5 Reactivation, shell syntax and leaving the environment

Each new Command Prompt needs activation before using the abbreviated commands in this document:

```
cd /d "C:\DOWNLOADS\cad-epsg-conversion"
call .venv\Scripts\activate.bat
python script.py
deactivate
```

Run `deactivate` after the application or build finishes. Closing the shell also ends its activation. An environment is a local development installation, not a portable distribution; recreate it rather than copying it to another machine.

Shell	Activation command	Continuation
Windows Command Prompt, used throughout this document	<code>call .venv\Scripts\activate.bat</code>	Caret <code>^</code> at the end of each continued line
PowerShell	<code>.\.venv\Scripts\Activate.ps1</code>	PowerShell backtick, or place the command on one line

The multi-line commands below are written for **Command Prompt**. Do not paste them unchanged into PowerShell. If PowerShell policy prevents activation, use Command Prompt with the documented `.bat` command. Calling an activation script from inside a Python subprocess cannot change the parent shell's environment; activation belongs in the shell before `python script.py`.

10.6 DWG support and ODA File Converter

Install a Windows-compatible ODA File Converter from the Open Design Alliance when DWG is required. It is a native format-conversion program with GUI and command-line interfaces. Neither `pip install ezdxf` nor installing a similarly named Python wrapper supplies its executable. [Open Design Alliance, ODA File Converter](#)

The script searches the explicit GUI/CLI setting, `ODA_FILE_CONVERTER`, `ODA_CONVERTER`, executable names on `PATH`, and supported Windows Program Files installation patterns. If detection fails, browse to the executable in Advanced settings or supply `--oda`.

Each ODA call receives an isolated copied input, requests `ACAD2018` output, disables recursion and enables auditing. There is no CAD-release selector in the application. ODA supplies the format bridge; PROJ performs the geodetic work. Native DXF input is handled through its document version where the resulting entities are supported. [ezdxf, ODA support](#)

11. Syntax compilation and standalone executable

11.1 Meaning and scope of a standalone build

The build below is intended to produce a **single Windows executable with all Python dependencies, the GUI runtime, PROJ resources and all four transformation grids embedded**. After validation, DXF opening, preview, coordinate conversion and DXF output require no separate Python, pip, Pillow or grid-file installation on the destination computer. Windows, normal system facilities and access to the user's drawing files are still required.

DWG input, output and preview require ODA File Converter separately. The current program therefore provides self-contained DXF operation, with an external dependency for DWG. A fully self-contained executable covering both DWG and DXF requires a compatible redistributable DWG runtime, its integration and clean-machine testing. No pip command or PyInstaller flag adds that engine to this application. [Open Design Alliance, ODA File Converter](#)

Resource	Bundled content	Host requirement
Python interpreter and imported standard-library modules	Yes	No separate Python installation
Tcl/Tk, Tkinter and application GUI	Collected by PyInstaller's Tk hooks	Compatible Windows desktop environment
ezdxf, Pillow/PIL, NumPy, FontTools and supporting packages	Collected with their modules and required binaries	No pip installation
pyproj, PROJ binaries and coordinate-system database	Collected with pyproj resources and hooks	No separate PROJ installation
Four .gsb grids supplied at build time	Explicitly embedded by --add-data	No external grids for the corresponding operations
DWG decoding/encoding	No	ODA File Converter, detected or selected explicitly
Custom fonts, Xrefs, underlays and other drawing-specific assets	Not automatically embedded	Provide the assets needed by the drawing; font substitution can affect preview appearance
Output CAD files and JSON reports	Created at runtime	A writable output folder
Temporary runtime and conversion files	Created as required	Available temporary disk space and permissions

One-file packaging describes delivery, not execution entirely inside the executable. PyInstaller extracts bundled resources into a temporary runtime directory and normally removes them when the application exits. This behaviour can affect startup time and the grid paths recorded

in a report.

11.2 Syntax verification and build-tool installation

Build a Windows executable **on Windows**, using the active 64-bit Python environment with compatible package wheels. A Linux build produces a Linux executable; PyInstaller is not a Windows cross-compiler. Start from a clean project environment containing the required packages from Section 10, then keep it activated:

```
python -m py_compile script.py
python -m pip install --upgrade ^
    "PyInstaller>=6.15,<7" pyinstaller-hooks-contrib
python -m pip check
python -m PyInstaller --version
```

`py_compile` checks source syntax and writes bytecode in `__pycache__`. It does **not** produce a standalone executable, collect dependencies or exercise the GUI. PyInstaller supplies the packaging step. The hooks package provides additional third-party collection rules and should be installed together with the build tool.

Keep this environment limited to the application requirements. Broad collection of ezdxf modules can otherwise pull in optional backends that happen to be installed. A build log can mention unavailable optional drawing backends; investigate warnings affecting this program's Tk/Pillow/font/PROJ path rather than installing unrelated GUI frameworks merely to silence every optional-import warning.

11.3 One-file GUI build with embedded grids

Before building, the current project folder must contain these files:

File	Build role
<code>script.py</code>	Main application, version 1.0
<code>pt73_e89.gsb</code>	Renamed DGT Datum 73 to ETRS89 grid
<code>ptLX_e89.gsb</code>	Renamed DGT Lisbon datum to ETRS89 grid
<code>ptED_e89.gsb</code>	Selected ED50 to ETRS89 grid
<code>ptLB_e89.gsb</code>	Selected Lisbon 1890 to ETRS89 grid

The command below embeds the four canonical files listed above. `pt73_e89.gsb` contains the renamed `D73_ETRS89_geo.gsb` data and `ptLX_e89.gsb` contains the renamed `DLX_ETRS89_geo.gsb` data. No additional copies under the download names are required. Renaming does not modify the datum, precision or binary contents. Preserve the files' provenance and checksums.

Run in the activated environment, from that folder:

```
python -m PyInstaller --noconfirm --clean --onefile --windowed --noupX --name script
↪ --collect-all ezdxf --collect-all pyproj --collect-all PIL --collect-all fontTools
↪ --add-data "pt73_e89.gsb;." --add-data "ptLX_e89.gsb;." --add-data
↪ "ptED_e89.gsb;." --add-data "ptLB_e89.gsb;." script.py
```

The output is `dist\script.exe`. Each quoted `--add-data` value uses the Windows `SOURCE;DESTINATION` form; `.` places the grid at the bundled resource root. All four files must

exist when building. The command is one line and can be pasted into Windows Command Prompt or PowerShell as shown.

Table 13. Build options and resource collection.

Option / mechanism	Purpose
<code>--onefile</code>	Deliver one application executable containing the selected resources
<code>--windowed</code>	Use a GUI executable without a normal console window
<code>--name script</code>	Produce <code>script.exe</code>
<code>--collect-all ezdxf</code>	Collect CAD modules, resources and package binaries
<code>--collect-all pyproj</code>	Collect pyproj modules, resources and binaries, with its dependency hooks
<code>--collect-all PIL</code>	Collect Pillow using its import name , including image plugins and package binaries
<code>--collect-all fontTools</code>	Collect FontTools using its case-sensitive import name, including dynamically accessed modules
Standard/contributed hooks	Handle additional dependency resources, NumPy native libraries and Tcl/Tk collection
Four <code>--add-data</code> arguments	Embed the actual NTV2 files used by the application
<code>--noupX</code>	Do not use UPX executable compression
<code>--clean</code>	Clear the build cache before packaging
<code>--noconfirm</code>	Permit replacement of the build output without an interactive confirmation

Pillow’s pip name is `Pillow`; its import and PyInstaller collection name is `PIL`. Similarly, pip installs `fonttools`, while collection uses `fontTools`. These names are intentional. Collection flags request the resources; the acceptance checks below establish whether a particular produced executable contains everything required on the target machine.

11.4 Embedded grids, overrides and deployment contents

For the embedded-grid deployment, distribute `dist\script.exe` with the English documentation and applicable grid attribution. The README supplies operating instructions and provenance; it is not loaded as a runtime dependency. External `.gsb` files and the `.venv` folder are **not required** for the embedded operations.

Leave **Advanced > NTV2 grid folder** blank to use the default search. In a frozen application, matching external grids beside `script.exe` have priority over embedded resources. An explicitly selected grid folder is exclusive; a stale folder selection can therefore prevent use of otherwise valid embedded grids. Do not set a development-machine grid path on the destination computer.

The embedded files are fixed at build time. To deploy different embedded grids, rebuild with the chosen files. For an intentional external override, place the correctly named grids beside the

executable or select a complete grid folder, and check the report's resolved paths and hashes. In either case, preserve DGT attribution and the grid provenance described in this paper.

Embedding the grid data has no effect on ODA discovery. Adding an arbitrary ODA executable to a build's data files is not a verified portable DWG deployment; its runtime dependencies and the application's discovery behaviour must also be addressed. The documented build uses an independently installed ODA converter for every DWG path.

11.5 Console executable for command-line operation and diagnostics

For an executable that displays command-line help, errors and progress, omit `--windowed` while retaining **all package collection and embedded-grid options**:

```
python -m PyInstaller --noconfirm --clean --onefile --noup --name script
↪ --collect-all ezdxf --collect-all pyproj --collect-all PIL --collect-all fontTools
↪ --add-data "pt73_e89.gsb;." --add-data "ptLX_e89.gsb;." --add-data
↪ "ptED_e89.gsb;." --add-data "ptLB_e89.gsb;." script.py
dist\script.exe --help
dist\script.exe "survey.dxf" --output-dir "output" ^
    --source-epsg 27493 --target-epsg 3763 --format dxf
```

Both build commands produce `dist\script.exe`; choose the intended variant before distribution because the second build replaces the first. The console variant can also launch the GUI when run without input drawings. If a windowed build fails before displaying its GUI, a console build from the same environment helps expose startup exceptions.

11.6 Reproducibility and clean-machine acceptance

Check source syntax and the application version before packaging:

```
python -m py_compile script.py
python script.py --version
```

The source-only delivery does not depend on a separate test suite or benchmark program. The executed development checks are summarized in Section 7.5. Before project deployment, convert a representative drawing with independent control points and inspect both the JSON report and the output in the intended CAD application. Test DWG input/output separately with ODA installed. Replacing the Python source or README does not update an existing `script.exe`.

Capture the resolved environment after installing both runtime and build dependencies:

```
python -m pip freeze --all > build-requirements.txt
python -c "import pyproj; pyproj.show_versions()" ^
    > build-environment.txt
python -c "import sys; print(sys.version)" ^
    >> build-environment.txt
python -c "import platform; print(platform.platform())" ^
    >> build-environment.txt
python -c "import platform; print(platform.machine())" ^
    >> build-environment.txt
python -m PyInstaller --version >> build-environment.txt
certutil -hashfile dist\script.exe SHA256
```

Retain the build files, Windows/Python architecture, package versions, executable hash and grid hashes with the release record. `pip freeze --all` records installed versions, including pip;

it is an environment snapshot rather than a dependency solver's lockfile. It does not capture the interpreter installer, OS, external ODA installation, custom fonts or grid data. On a separate build machine, recreate and activate the environment, bootstrap pip, then run `python -m pip install -r build-requirements.txt` and supply the recorded grids before repeating the build.

The script isolates the external converter's library-search environment when launched from a frozen application, including restoration of the Windows DLL search directory. This is necessary because the bundled Python runtime and the external ODA process can require different libraries.

Acceptance check	Expected evidence
Run only the executable, without Python, a virtual environment or external <code>.gsb</code> files	GUI opens with the input controls visible and the preview available
Open a DXF containing lines, curves, text and hatches	The black preview renders representative content; Fit, pan and zoom work
Convert known Datum 73 control to EPSG:3763 with the grid folder blank	Output and <code>survey_EPSG3763.json</code> are produced; report identifies the embedded grid and its expected hash
Check reverse and other required datum paths	Selected operations resolve the intended embedded grid files and agree with control within the project's acceptance criteria
Test the local-grid DXF workflow without an internet connection	No package or grid download is needed for the supplied operations
Use paths containing spaces and a writable output folder	CAD output and report creation succeed
Review the output in the intended CAD application	Relevant geometry, layers, text and complex entities meet the documented scope and tolerances
Install/select ODA and test DWG input and output separately	ODA runs successfully and the resulting files reopen in the intended CAD software

These are deployment acceptance steps, not a claim that a particular Windows executable has already passed them. Source syntax, package imports and coordinate tests cannot replace execution of the frozen program on the target platform. A successful native build plus these checks is the evidence for a working standalone DXF distribution.

12. Reports, reproducibility and troubleshooting

12.1 Output names and report lifecycle

A default input `survey.dxf` produces the following file, according to the selected output format:

- DXF output: `survey_EPSG3763.dxf`.

- DWG output: `survey_EPSG3763.dwg`.

For this input the report is `survey_EPSG3763.json`. In a batch, the filename uses the **first input's stem** and the target EPSG: `<first-input-stem>_EPSG<target EPSG>.json`. The report contains all completed batch files; it is not a separate JSON file for every input.

A conversion that finishes after omitting unprocessable geometry is reported as `completed_with_omissions`. The output drawing contains the geometry that was successfully processed; it must not be interpreted as a complete reproduction of the source. The `omitted` counters include all omitted objects. General failures have `entity_omitted` issues; grid-domain skips have `outside_grid_object_skipped` issues, per-type `skipped_outside_grid` counters and structured `outside_grid_objects` records. These records identify source handles and coordinates without reclassifying them as successfully transformed objects. Keep the original drawing and the report with the converted output.

A completed run with a drawing-level notice or boundary-only hatch fallback is marked `completed_with_warnings` when there are no omissions. Generic WGS 84/epoch and explicitly degraded operation notices are visible in the report rather than hidden behind a clean success label. An ordinary `completed` status does not mean there were no local CAD approximations; inspect entity issues.

Each run replaces the report for the same first-input stem and target in the selected output folder, independently of the CAD overwrite setting. Archive the report elsewhere if separate run records are required. Existing CAD outputs are replaced only when overwrite is enabled. Output/input collisions and duplicate planned output names are rejected before processing.

If a batch stops on an error or cancellation, completed drawings remain available. The report records the batch state and completed files when processing has started. An initial validation failure can occur before report creation. The `files` list describes completed drawings. When per-file processing has started, `failed_file` records the current drawing's collected counters, issues, handles and source archives. These are partial diagnostic data, not evidence that the failed drawing was fully transformed or published. The `error` entry identifies the failure and input path.

12.2 Report contents

Table 14. Main report information.

Report area	Information
Application/job	Name, version, selected settings and input/output paths
Execution	Start/finish UTC timestamps, completed count and batch status
Coordinate operation	Source/target identifiers, names, operation description and pipeline
Grid provenance	Actual filename, resolved path, SHA-256, byte size,

Report area	Information
	internal version, datum identifiers, direction and
	all validated subgrid bounds, ellipsoids and node counts
CAD processing	Transformed, approximated, local-affine, exploded
	unresolved and omitted counters
Entity issues	Severity, issue code, message, layout, entity type and handle
Read/write assurance	Input audit/recovery and output reopening/audit/count checks
Failure/cancellation	Error type, message and current input path where available
Failed drawing	Partial counters, issues, entity handles and source
diagnostics	archives in <code>failed_file</code> , when available

The `environment` records Python, ezdxf, pyproj, PROJ and EPSG database versions and the disabled-network state. `coordinate_operation.stages` identifies each projection/datum stage, its pipeline, stated accuracy, grid resources and coverage. `files[].numerical_checks` gives sampled forward/inverse closure, source/target tolerance settings and the number of one-sided derivative probes. `output_verification.coordinate_comparison` records serialization discrepancies and its explicitly checked scope. Small closure is not a replacement for independently known coordinates.

For a local-grid chain, `accuracy_metres` is deliberately `null`. The script does not replace it with a guaranteed centimetre value or automatically insert DGT's validation RMSE. For generic PROJ chains, reported stage accuracy information is descriptive metadata; it is not the covariance propagation described in Section 7.1.

With DGT originals or compatible aliases, look for the actual internal version IGP2011 and the intended file hash. The name `pt73_e89.gsb` alone does not identify which set of binary corrections was used.

Polyline-specific issue codes distinguish deliberate handling from an unexplained loss: `empty_polyline_omitted` identifies a zero-vertex lightweight polyline omitted by design; `empty_polyline_preserved` identifies an empty legacy polyline; `zero_length_bulge_cleared` identifies coincident vertices with an unusable bulge; `polyline_width_local_rescaled` records direction-dependent local width/thickness treatment; and `polyline_representation_changed` identifies conversion to a 3D representation. The transformed counter `LWPOLYLINE_EMPTY_OMITTED` counts omitted empty objects; it does not count reprojected vertices. Inspect issue handles when reconciling the source and output drawings.

For each fitted polyline materialized as an ordinary polyline, the completed file's `fitted_polyline_sources` list records its source and output handles, source EPSG, DXF

version, fit flags, display/auxiliary vertex handles and original DXF tag text in `source_dxf`. The archived coordinates remain in the source CRS. Keep this report with the output drawing when the original fitting data is needed, and archive it before another run replaces `survey_EPSG3763.json`. The associated issue code is `fitted_polyline_faceted`.

Proxy replacements are recorded as `proxy_graphics_materialized`, with source data in `proxy_entity_sources`. Replacement handles identify native entities created during preparation; later faceting or block explosion may replace or copy them again. In the default mode, `entity_omitted` records unsupported proxy geometry and its removal from the processed output, with its handle and recovery guidance. Under strict mode, `proxy_unresolved` identifies the problem, the error shows the first issue, and `failed_file.issues` retains all issues collected before failure. A rejected composite may cause the containing INSERT to be recorded as omitted or unresolved.

For omissions, `handle` identifies the object being processed, while `source_handle`, `source_entity_type` and `source_layout` identify its originating object in the input drawing. For a generated primitive this can be its parent INSERT, DIMENSION or proxy. The batch field `omitted_entity_count` sums omitted work-copy objects across published files; it is not a count of unique original handles or visible instances. For example, an unsupported member removed from a reused block definition is counted once, even if that definition has multiple insertions.

For a proxy carrying plot-style, material or mapper commands, read the appearance notes in its `proxy_graphics_materialized` warning. They identify styling retained only in the source archive. The same warning identifies a proxy lineweight replaced with ByLayer when its saved value is outside the supported range. A successful geometric conversion does not certify identical plot styles or photorealistic rendering.

Successful auxiliary cleanup. `auxiliary_cleanup_count` counts affected objects whose invalid optional coordinates were removed after their boundary geometry passed the full operation. Original seed positions and reasons remain in `auxiliary_coordinate_repairs`, and `invalid_hatch_seed_removed` is recorded as information. The completion dialog reports these as completed cleanup, separately from drawing/display warnings and omissions. A valid-location coordinate-operation error is not downgraded to cleanup. A clean completion label still does not certify the absence of local CAD approximations.

Unresolved source evidence. Failed ELLIPSE and ACAD_PROXY_ENTITY definitions are saved in `unresolved_entity_sources`, with handle, layer, layout, source EPSG, reason and DXF tags when obtainable. Proxies also record the stored graphics byte count and hash. Archived source coordinates are not target geometry and are not inserted into the converted drawing. A proxy without a usable display representation remains an explicitly reported omission; no rectangle, origin marker or fictitious curve is substituted. [ezdxf](#), [Proxy entity](#)

12.3 Temporary files and cancellation

DWG exchange and output verification use isolated temporary directories. Completed CAD output is staged in the destination filesystem before publication. Normal completion, handled errors and cooperative cancellation clean the temporary working data.

Cancellation is checked between processing steps and entities. Some expensive operations must return before the next check; cancellation need not be instantaneous. When ODA is active,

the application attempts to stop its process tree and clean the associated workspace. Abrupt operating-system termination is outside the normal cleanup sequence.

12.4 Diagnostic guide

A **Saved vertex count changed** message from an earlier source is generated during output verification, not by the preview renderer. The corrected source handles the equivalent text-field and exact-closure cases in Section 7.4 without relaxing coordinate accuracy. If a saved-geometry error remains, inspect `failed_file.output_verification`: it identifies the failed entity and processing stage, with counts or positions when available. Do not change EPSG settings or relax the geometric tolerance solely to hide this error. Replace the source or rebuild the executable, then rerun from the original drawing.

Table 15. Common symptoms and checks.

Symptom	Issue / next check
No module named PIL, or preview dependency error from source	Activate the project environment, install Pillow and the complete Section 10 runtime requirements, then repeat the drawing-frontend import check
Preview dependency error from <code>script.exe</code>	Rebuild with Pillow installed and <code>--collect-all PIL</code> ; installing pip packages beside a frozen executable does not repair its bundle
Tkinter demonstration fails before the program starts	Check the Python installation's Tcl/Tk components; Tkinter is not a pip package
A Command Prompt command reports redirection or caret errors	Quote version constraints and remove trailing spaces after carets; use the specified shell
Executable cannot use its embedded grid	Clear the explicit grid-folder setting, check for unintended external overrides and inspect the recorded grid path
Required NTV2 grid is missing	Check a recognised datum-specific filename and the exclusive/default search rules
DGT files are present but not found	Check the selected grid folder. Original DGT names and legacy aliases are recognised; no renaming is required.
Unexpected datum or units in grid header	A file was assigned to the wrong datum family; check binary provenance
Grid coverage error	Check the source CRS, units and failing-object coordinates in the report. Cleanup is on by default; strict mode or <code>--stop-outside-grid</code> turns a domain error into a stop.
Very large or implausible displacement	Check source datum, false origin, axis order and source drawing units
EPSG:2963 result is	Check X=southing/Y=westing and the

Symptom	Issue / next check
inconsistent DWG conversion cannot start	historical Bessel–Bonne realisation Check ODA installation, executable path and operating-system compatibility
Output exists	Select another folder/suffix or deliberately enable overwrite
Two drawings create one output name	Rename one input or process them separately
Strict mode stops a block/dimension	Clear the strict option to finish with reported omissions, or repair the object in the authoring application
Proxy object cannot be converted	Default processing omits it and continues. Inspect <code>proxy_unresolved</code> , <code>entity_omitted</code> and the handle; export standard entities from its authoring application to recover it
Proxy lineweight is outside the supported range	A valid geometry snapshot uses <code>ByLayer</code> and reports the appearance substitution
Conversion completes with omissions	Inspect <code>omitted</code> counts and issues; compare with the unchanged source drawing
Dimension text differs from a measured target length	Rendered text was transformed, not recalculated as a target-system measurement
Preview looks incomplete	Read preview notices; display limits do not define conversion limits
Preview still shows source coordinates after conversion	The preview is the selected original input; open the output separately
No CLI text from <code>script.exe</code>	Use the console build or run the Python source
Elevations appear unchanged	Expected for horizontal conversion with Keep Z enabled; no geoid correction is applied

13. Discussion and conclusions

The principal geodetic decision is the correct identification of the source system. A precise numerical engine applied to the wrong datum, projection, units or axes can produce a consistent but incorrect drawing. For mainland work requiring the national target system, the documented workflow centres on EPSG:3763 and uses the corresponding historical-datum grid when necessary.

Grid transformations represent spatially varying datum differences that a single national translation or similarity transformation cannot fully describe. Their usefulness depends on the quality and coverage of the underlying observations, not simply on file size. Recording the grid’s version and hash makes a transformation reproducible and permits later comparison with independent control.

CAD introduces a second problem beyond geodesy: the target geometry may not belong to the same analytical or parametric object class as the source. The application addresses this

through explicit geometric strategies and records approximations, unresolved conditions and omissions. Its default best-effort policy completes processable geometry while omitting objects that cannot be transformed safely; strict mode is available when identified omissions must prevent publication. It does not claim universal exact preservation of every proprietary object, embedded coordinate payload or associative editing relationship.

A defensible project deliverable combines three forms of evidence: a documented coordinate-operation chain, independent agreement with suitable control points, and inspection of the CAD objects that carry engineering meaning. The JSON report supports the first and records structural processing information; the remaining evidence comes from the project’s survey and CAD review.

14. Bibliography

Titles of Portuguese-language resources are translated into English below. Institutional web pages without a stated publication date are identified as *n.d.* Links identify the technical references used by the application documentation. The equations are explanatory formulations in this document’s notation; citations distinguish institutional specifications, methodological literature and software documentation.

14.1 Portuguese reference systems and DGT specifications

1. **Direção-Geral do Território (DGT).** n.d. [Reference systems](#). Geodesy documentation, Portugal. Primary overview of mainland and island reference systems and national/island vertical datums. Portuguese-language institutional resource.
2. **Direção-Geral do Território (DGT).** n.d. [PT-TM06/ETRS89](#). Definition, establishment and projection parameters of the mainland national system. Principal institutional reference for the EPSG:3763 workflow. Portuguese-language institutional resource.
3. **Direção-Geral do Território (DGT).** n.d. [Coordinate transformations: mainland Portugal](#). NTv2 grid construction and validation, downloadable Datum 73/Datum Lisboa grids, and parameter-transformation alternatives. DGT states a Creative Commons Attribution 4.0 International licence for the grid work. Portuguese-language institutional resource.
4. **Direção-Geral do Território (DGT).** n.d. [PTRA08-UTM/ITRF93](#). Reference-system establishment and UTM parameters for the autonomous regions. Portuguese-language institutional resource.
5. **Direção-Geral do Território (DGT).** n.d. [Datum 73](#). Historical mainland network, origin and Hayford/Gauss–Krüger parameters. Portuguese-language institutional resource.
6. **Direção-Geral do Território (DGT).** n.d. [Datum Lisboa](#). Historical mainland system and Local Lisbon Triangulation context. Portuguese-language institutional resource.
7. **Direção-Geral do Território (DGT).** n.d. [Bessel Datum Lisboa](#). Bessel–Bonne parameters, west/south coordinate directions and the qualification concerning historical polynomial derivation. Portuguese-language institutional resource.
8. **Direção-Geral do Território (DGT).** n.d. [ED50: European Datum 1950](#). Datum/ellipsoid context and UTM representations relevant to Portuguese data. Portuguese-

language institutional resource.

9. **Direção-Geral do Território (DGT)**. n.d. [Geoid model](#). GeodPT08, the relationship between ellipsoidal and orthometric heights, and available geoid datasets. Portuguese-language institutional resource.
10. **Instituto Geográfico Português (IGP), hosted by DGT**. 2009. [Transformation parameters for mainland Portugal](#). Parameter sheet dated July 2009; degree-2 polynomial, Bursa–Wolf and Molodensky parameters and residual statistics. Portuguese-language PDF.
11. **Direção-Geral do Território (DGT)**. n.d. [Formula sheet: second-degree polynomial transformation](#). Normalized-coordinate equations and coefficient definitions. Portuguese-language PDF.
12. **Direção-Geral do Território (DGT)**. n.d. [Formula sheet: Molodensky transformation](#). Geographic-coordinate and ellipsoidal-height equations, ellipsoid differences and curvature radii. Portuguese-language PDF.
13. **Direção-Geral do Território (DGT)**. n.d. [Formula sheet: Bursa–Wolf transformation](#). Small-angle transformation matrix, parameter definitions and rotation-convention qualification. Portuguese-language PDF.

14.2 Scientific and geodetic methodology

14. **Gonçalves, J. A.** 2009. [Conversions of national coordinate systems to ETRS89 using grids](#). Paper presented at the VI National Conference on Cartography and Geodesy, CNCG 2009. Faculty of Sciences, University of Porto. Portuguese-language conference paper; the project-supplied PDF was examined, including its independent validation results.
15. **Gonçalves, J. A.** n.d. [Coordinate transformations in Portugal](#). Faculty of Sciences, University of Porto. Portuguese-language technical resource describing four historical-datum NTv2 grids and their use with PROJ; consulted through the project-supplied PDF copy.
16. **International Association of Oil & Gas Producers (IOGP)**. 2019. [Geomatics Guidance Note 7, Part 2: Coordinate Conversions and Transformations including Formulas](#). Report 373-7-2, September 2019 edition. Reference for operation terminology and mathematical methods, including grid interpolation and inverse transformations. This citation identifies the particular edition consulted.
17. **Snyder, J. P.** 1987. [Map Projections—A Working Manual](#). United States Geological Survey Professional Paper 1395, ix + 385 pp. Washington, DC: United States Government Printing Office. DOI: 10.3133/pp1395. Classical further reading on ellipsoidal map projections, their parameters and distortion.
18. **Joint Committee for Guides in Metrology (JCGM)**. 2008, corrected 2010. [Evaluation of Measurement Data—Guide to the Expression of Uncertainty in Measurement](#). JCGM 100:2008. Sections 5.1–5.2 provide the general first-order framework for uncertainty propagation, including correlated inputs.

14.3 Coordinate and CAD software

19. **PROJ contributors.** n.d. [Horizontal grid shift](#). PROJ operation documentation. Grid-based horizontal corrections, supported grid formats and optional-grid behaviour.
20. **PROJ contributors.** n.d. [Geodetic TIFF grids](#). PROJ grid-format specification, especially longitude-offset sign conventions and their relationship to NTV2.
21. **PROJ contributors.** n.d. [Geodetic to Cartesian conversion](#). PROJ operation documentation. Geographic/geocentric coordinate representations and Cartesian axes.
22. **PROJ contributors.** n.d. [Transverse Mercator](#). PROJ projection documentation. Mathematical formulation, parameters and precise numerical implementation.
23. **PROJ contributors.** n.d. [Helmert transform](#). PROJ operation documentation. Three-/seven-parameter and time-dependent forms, units and rotation conventions.
24. **pyproj contributors.** n.d. [Transformer API](#). pyproj documentation. Coordinate-operation construction, `always_xy`, operation availability and accuracy metadata.
25. **ezdxf contributors.** n.d. [ODA File Converter support](#). ezdxf 1.4.4 documentation. External converter integration and DWG/DXF exchange context. The application uses its own controlled subprocess workflow.
26. **Open Design Alliance.** n.d. [ODA File Converter](#). Product and download documentation, including supported command-line conversion inputs.
27. **pyproj contributors.** n.d. [Advanced transformation examples](#). Area-of-interest selection, missing grid operations and explicit promotion of CRS definitions to 3D.
28. **pyproj contributors.** n.d. [PROJ network settings](#). Explicit control of local versus network-accessible transformation resources.
29. **Direção-Geral do Território (DGT).** n.d. [National EPSG code list](#), in Open Data. Identifies the shared geographic/geocentric PTR08 codes for the Azores and Madeira.
30. **PROJ contributors.** n.d. [EPSG-derived projected CRS definitions](#) and [coordinate conversion definitions](#). Primary software database definitions for the supported CRS names, datums, projections, axes and areas of use.
31. **ezdxf contributors.** n.d. [Zoom Layouts](#). Saved viewport fitting, single-window configuration and dependence on the receiving CAD window's aspect ratio.
32. **ezdxf contributors.** n.d. [HATCH entity reference](#). Boundary paths, object-coordinate-system geometry and stored seed points.
33. **ezdxf contributors.** n.d. [Text entity and placement API](#). Active insertion/alignment positions, FIT/ALIGNED endpoints and world-coordinate interpretation through the object's coordinate system. Consulted on 18 September 2026 for the saved-geometry verification fix.